

Evaluating Code Generation for Industry-Specific Software Solutions: A Comparative Study

1. Abstract

Many industries lack tailored software solutions that address their unique pain points. Large Language Models (LLMs) have the potential to generate high-quality, domain-specific code with integration-ready capabilities. However, current LLMs' outputs often fall short in terms of best practices, modularity, security, and documentation standards. There is a need to guide LLMs to Produce specialized, Professional-grade software across diverse industries. This study aims to evaluate LLM-generated code against established software industry standards and best practices.

We performed human preference evaluations for code generation using state-of-the-art (SOTA) models, such as **OpenAI o4-mini-high**, **Gemini 2.5 Pro**, and **Claude Opus 4**, on Programming tasks in **C++**, **Python**, **Java**, and **JavaScript**. These Prompts are derived from industry-specific software solutions. A comprehensive human evaluation was conducted using a detailed rubric, based on **Relevance**, **Completeness**, and **Correctness**, scoring each model's response across 40 diverse Programming Prompts, within various industries, such as Gaming, FinTech, IoT, Machine Learning, etc.

The golden responses were generated using open-source LLMs and refactored by experts according to software-industry best practices, with a focus on **security**, **reliability**, **redundancy**, and **maintainability**. For system evaluation, this report uses **CodeBLEU** to assess alignment based on **BLEU N-Gram Match**, **Semantic Data-Flow Match**, and **Syntactic AST Match**. This combined approach of human-in-the-loop assessment and automated evaluation provides a comprehensive analysis of SOTA models against industry-grade code and serves as a foundation for training and aligning SOTA code generation models.

OpenAI o4-mini-high excels when precision and reliability matter most delivering syntactically correct, tightly scoped solutions that hit every bullet in the prompt. Its strength lies in quickly producing lean, dependable prototypes for domains like IoT and FinTech where strict adherence to constraints is paramount. However, this laser focus comes at the expense of broader completeness and architectural richness: users will frequently need to augment its outputs with production-grade error handling, extensibility patterns, and deeper domain-specific logic to match the sophistication of competitors like **Claude Opus 4** or **Gemini 2.5 Pro**.

2. SOTA Model Selection

We selected the following SOTA language models based on recency and ranking on popular leaderboards focused on coding [1].

- **OpenAI o4-mini-high** is a premium variant of the o4-mini model, specifically optimized for coding tasks with enhanced accuracy and faster Processing. It demonstrates exceptional performance in algorithmic Problem-solving and code generation.
- **Gemini 2.5 Pro** is Google's advanced thinking model with exceptional coding capabilities. It Processes entire codebases and generates high-quality code across multiple languages with a sophisticated understanding of software architecture patterns.
- **Claude Opus 4** is Anthropic's flagship reasoning model from the **Claude Opus 4** family, designed for complex analytical tasks and sophisticated Problem-solving across diverse domains.

3. Dataset

We curated a diverse set of 40 industry-specific Programming Prompts, covering a wide spectrum of real-world application domains. These Prompts span four major Programming languages in the following domains:

- **C++:** Gaming, IoT, and Security
- **Python:** Machine Learning, FinTech, and EdTech.
- **Java:** E-Commerce, CRM, and Hotel.
- **JavaScript:** Social Networking, Media Publishing, and Video Streaming.

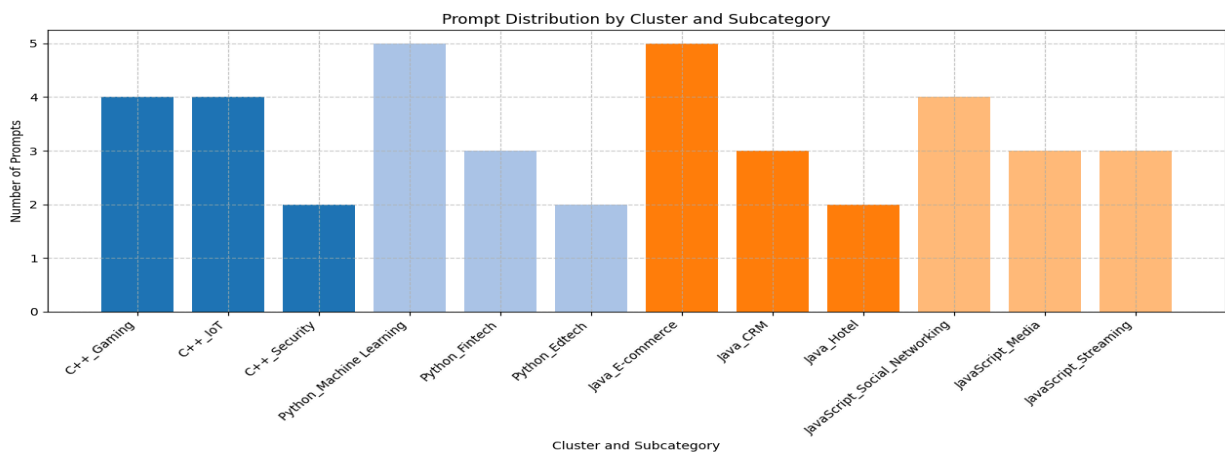


Figure 1. Industry-specific Prompt distribution plot.

Language	Industry	Sub Topic
C++	Gaming	Simple 2D Physics Engine
		Procedural Level Generator
		Command-Line Multiplayer Lobby
		Shader Demo with OpenGL
	IoT	MQTT Temperature Publisher
		GPIO-Based Home Alarm
		Over-The-Air (OTA) Update Simulator
		Edge Analytics with SQLite
	Security	Password Strength Estimator
		Simple TLS Client
Python	Machine Learning	Time-Series Anomaly Detection
		Few-Shot Text Classification
		Image Captioning
		Reinforcement Learning for Grid Navigation
		Unsupervised Representation Learning
	FinTech	Portfolio Optimization
		Option Pricing
		Fraud Detection
	EdTech	Adaptive Quiz Engine
		Knowledge Graph Recommendations
Java	E-Commerce	SKU Demand Forecasting
		Order Confirmation PDF Generator
		Real-Time Stock Availability Polling
		Shopping Cart Price Rules Engine
		Coupon Code Generator & Validator
	CRM	Lead Scoring via Rule Engine
		Email Template Personalization
		Activity Timeline Aggregator
	Hotel	Room Availability Calendar
		Dynamic Pricing Engine
JavaScript	Social Networking	Client-Side Friend-Suggestion Algorithm

		Real-Time Chat via WebSockets
		Activity Feed Renderer
		Offline Draft Storage
	Media	Image Metadata Extractor
		Canvas-Based Audio Visualizer
		Client-Side Video Cutter
	Streaming	Adaptive Bitrate Player
		Live Comments Overlay
		Segmented Download & Play

Table 1. Industry-specific subtopics used for Programming Prompts

Each Prompt was independently written on the following criteria:

1. **Prompt constraints:** The Prompt should have at least three constraints.
2. **Domain expertise:** Prompt should have terms commonly used by domain experts and software engineers.
3. **Prompt length:** Restrict the Prompt to less than 150 words.

Examples:

**I lead data initiatives at a major energy utility. I need a Python engine for time-series anomaly detection on streaming data from smart meters, grid sensors, and operational logs. The system should catch abrupt changes, seasonal shifts, and multi-scale trends, flagging true anomalies while reducing false alerts from environmental noise. It must integrate seamlessly with our data pipelines using frameworks like PySpark, adapt automatically to evolving data without frequent retraining, and Provide root-cause explanations as short textual summaries or feature attribution scores for each alert. All code should use type hints, Google-style docstrings, and be structured for maintainability. The engine should be accessible for other services and support alerting via email and SMS. Full compliance with data privacy standards is required. The solution must be extensible for future sensor types and analytics. Return the code in a single file.*

**I'm a frontend engineer building a privacy-first mobile social app for low-connectivity regions. I need a single-file React module that shows friend suggestions using mutual followers, recent activity, shared interests, and common groups. It should be able to function in offline mode, with easily customizable ranking logic. Include optional peer-to-peer sync using Automerge over a secure connection. The UI should be fully*

responsive. Provide JSDoc comments and a concise integration guide tailored for a peer-to-peer, low-bandwidth app. Use the latest tech stack.

4. Evaluation criteria

Each model-generated response was manually evaluated by three human Programmers across three key criteria using a 1–5 point-wise ranking scale, where 1 indicates poor and 5 indicates excellent. In addition, a system-level evaluation is conducted using **CodeBLEU** to assess the code quality compared to a “Golden Response” generated by experts. The criteria include:

- **Relevance:** Alignment with the Prompt.
- **Completeness:** Coverage of all aspects of the query.
- **Correctness:** Accuracy and factual validity.
- **CodeBLEU:** System evaluation metrics (see appendix)

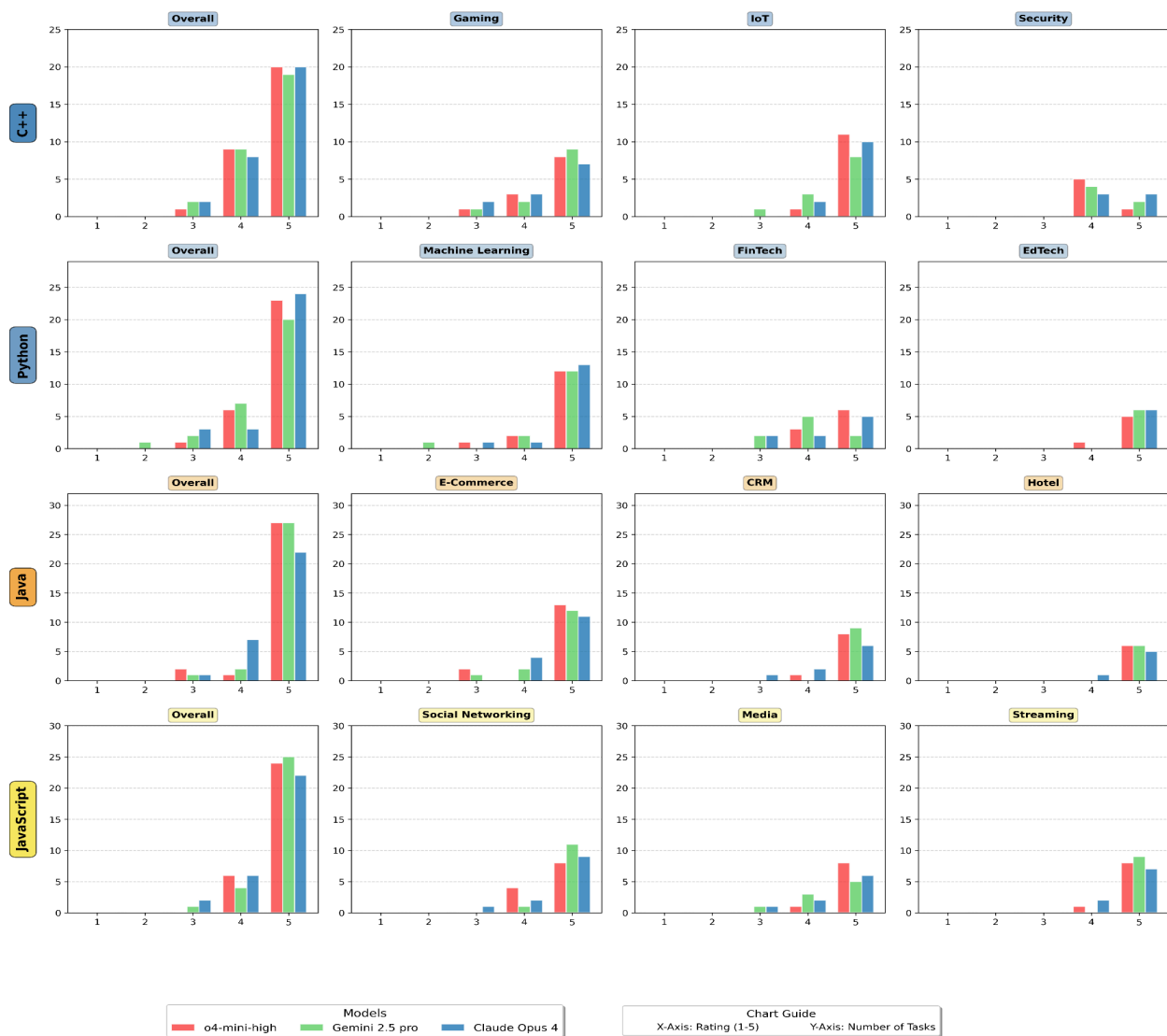
This comprehensive evaluation framework ensures a nuanced assessment of model outputs, capturing both technical accuracy and user-preferred quality dimensions.

Relevance

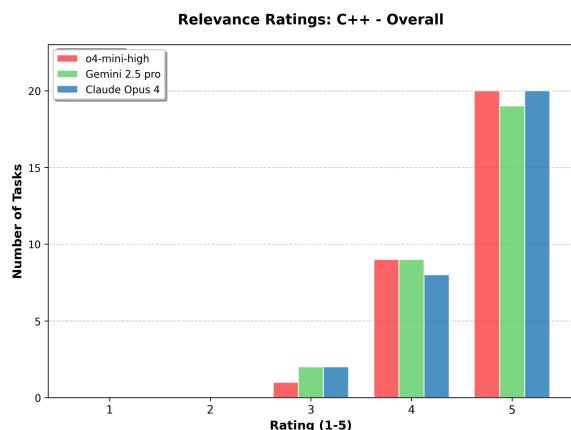
Relevance measures how closely an LLM's generated code aligns with the specific intent and requirements of a given task or Prompt. It's a measure of whether the code outputs:

- Directly addresses the Problem or feature requested in the Prompt.
- Adheres to all explicit constraints and specifications Provided in the Prompt.
- Avoids including extraneous or off-topic information not pertinent to the Prompt's request.

Relevance Ratings - Cross-Language Domain Comparison



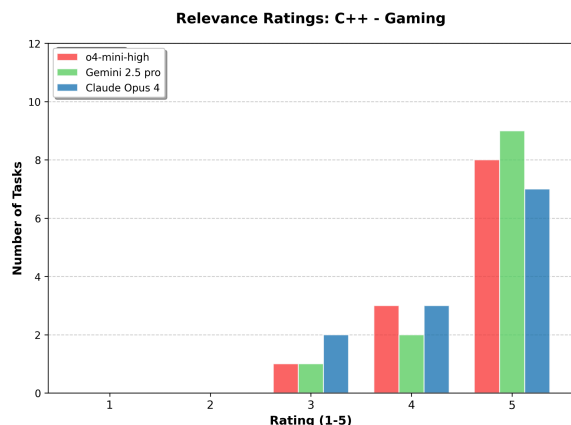
C++ Language



Overall

The graph shows that both **OpenAI o4-mini-high** and **Claude Opus 4** received the highest number of perfect relevance ratings (score of 5), with Claude also slightly ahead in consistency across ratings 3–5. However, **Gemini 2.5 Pro** trails slightly in total perfect scores, making **Claude Opus 4** and **OpenAI o4-mini-high** joint top performers, with **Claude** edging ahead in distribution smoothness.

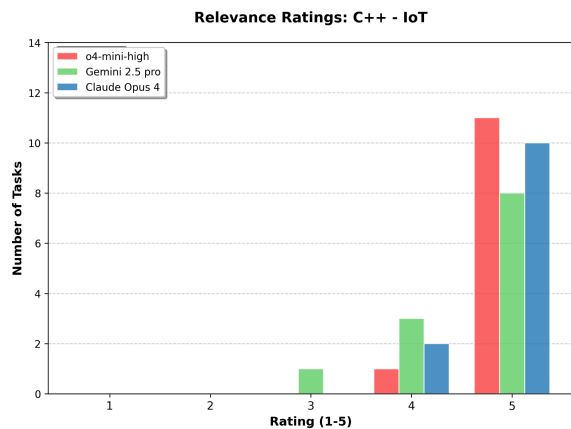
Gaming (C++)



Gemini 2.5 Pro stands out as the leader. It consistently demonstrated the most direct and focused alignment with the user's requests, securing the highest number of perfect scores. Even when its relevance was slightly penalized, it was for being "on-topic, but lean[ing] into explanation of Procedural generation patterns that weren't directly asked" a minor deviation. Its most significant relevance issue was for using "older C++ patterns despite modern requirements" which, while a valid critique, did not detract from addressing the core Problem.

OpenAI o4-mini-high is a very close second, achieving a high number of perfect scores. It generally shows a strong ability to adhere to the Prompt's requirements. However, it was marked down significantly in one instance for providing a solution that "misses large parts: collisions, visualization, etc." indicating a potential for major omissions. In other cases, its relevance was reduced because it spent "time explaining general Procedural generation rather than this specific code" or didn't use C++23 features significantly as requested.

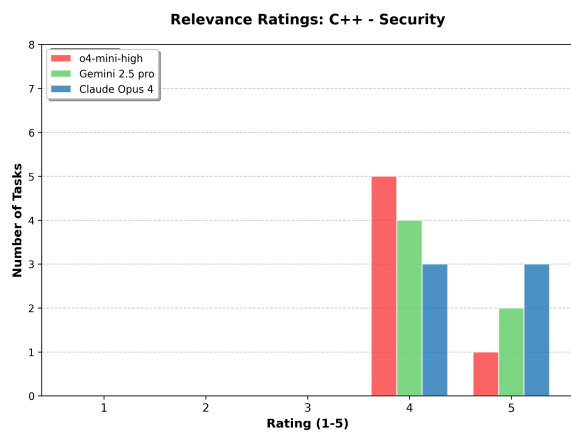
IoT (C++)



OpenAI o4-mini-high is the leader, demonstrating the most consistent and precise alignment with user requests by securing the most perfect scores. It repeatedly delivers solutions that perfectly match the complex requirements of IoT engineering Prompts. Even when it did not receive a perfect score, the justification noted it was for lacking more "Advanced router placement optimization algorithms," indicating it still fulfilled all the core, explicitly stated requirements.

OpenAI o4-mini-high excels at meticulously addressing all the complex, layered requirements common in IoT Prompts, from low-level hardware interaction to networking Protocols. However, its relevance can be slightly diminished when it provides a correct implementation of the stated requirements but doesn't include more sophisticated logic, such as "Advanced router placement optimization algorithms" that go beyond basic detection.

Security (C++)

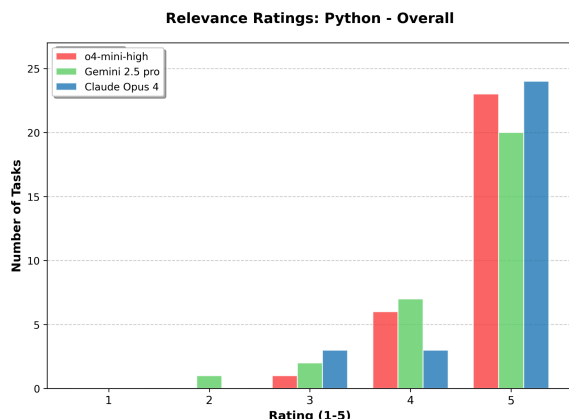


Claude Opus 4 stands out as the leader, securing the most perfect scores. It consistently demonstrates the best ability to create solutions that are highly aligned with the specific and demanding requirements of security-focused Prompts. Justifications for its perfect scores praise it for "directly" meeting the Prompt's needs for a "TLS 1.3, certificate validation, SNI, and latency use case. Even when its relevance was slightly reduced, it was for minor architectural choices like "header file structure could be imProved" or for not perfectly aligning with

"sub-millisecond performance goals," while still addressing the core problem effectively.

OpenAI o4-mini-high relevance can be high, but it is often comPromised by a failure to deliver the level of sophistication required for high-security applications. For instance, its password estimator used a "simplistic entropy model," which is not suitable for a fintech environment, and its TLS client didn't "fully implement structured logging," a key requirement for audit and compliance.

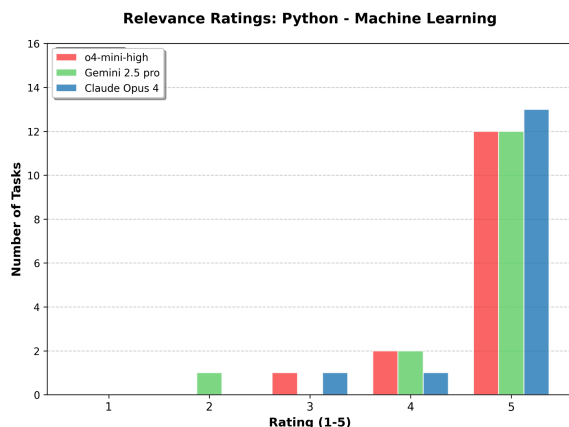
Python Language



Overall

Claude Opus 4 received the highest number of perfect relevance ratings (score of 5) for Python tasks, slightly outperforming **OpenAI o4-mini-high**, while **Gemini 2.5 Pro** lags in perfect score. Given its strong lead in top ratings and minimal low scores, **Claude Opus 4** is the clear winner in Python relevance overall.

Machine Learning (Python)



Claude Opus 4 emerges as the clear winner, achieving the highest number of perfect scores. It consistently demonstrates an exceptional ability to directly and comprehensively address all components of a user's request. Justifications for its perfect scores repeatedly state that "Every stated requirement... is explicitly addressed" and that it "Directly implement[s] all Prompt requirements." Even when not scoring perfectly, Claude's solutions were still considered highly relevant, only missing deeper elements like "PySpark integration and automated adaptation," but

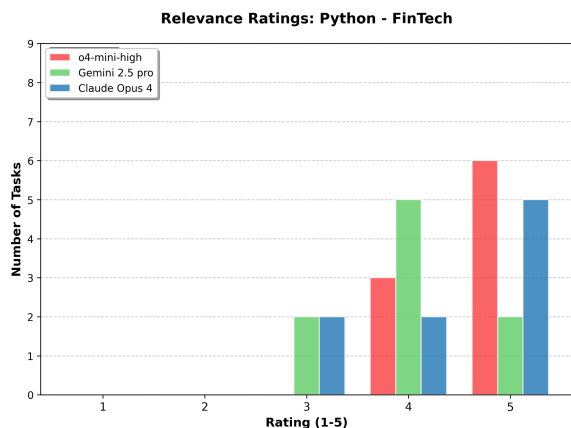
still addressing all core needs.

OpenAI o4-mini-high and Gemini Flash 2.5 are tied for second place. While both received a fair number of perfect scores for directly implementing all stated requirements, their relevance was more inconsistent. In particular, **OpenAI o4-mini-high's** relevance was significantly penalized when it misunderstood core concepts of the Prompt. This conceptual misalignment led to solutions that, while functional, were not relevant to the user's actual goal. For instance, in one case, it was marked as only "Partially aligned" because the "Core requirement of meta-learning is misunderstood." In another, its meta-learning component was described as "superficial rather than implementing genuine meta-learning algorithms," making the response less relevant.

OpenAI o4-mini-high's relevance suffers when it fails to grasp the deeper, more complex technical or conceptual nuances of a Prompt, even if it addresses the surface-level keywords. This results in solutions that appear to follow the instructions but are fundamentally misaligned

with the user's primary objective, such as treating a complex learning strategy as simple fine-tuning.

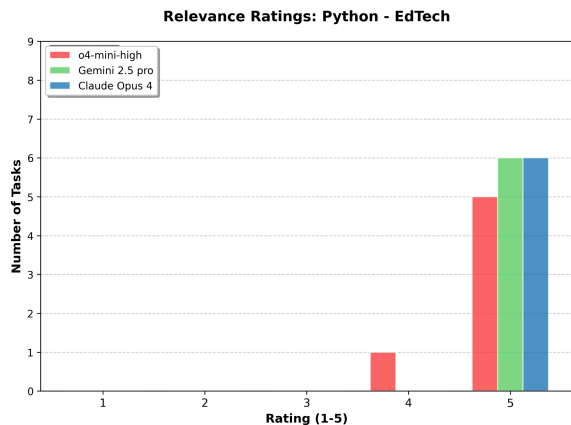
FinTech (Python)



OpenAI o4-mini-high is the top-performing model. It consistently demonstrated the most direct and complete alignment with the user's stated requirements. Justifications for its numerous perfect scores frequently emphasize that "Every element of the Prompt is addressed directly" and the solutions have "perfect alignment with the user's needs". Even when its score was slightly lower, the justification noted that it still addressed all "high-level requirements".

This indicates **OpenAI o4-mini-high's** primary strength is its precision in mapping its output directly to the explicit requests in the Prompt without introducing extraneous features or missing key components. Claude Opus 4 is a very close competitor, also receiving several perfect scores with justifications stating, "Every requirement from the Prompt appears in the code" and that its solution "maps exactly to what a regulated hedge-fund environment demands". However, its relevance was occasionally penalized when it over-delivered, adding features like extra database or caching layers that, while impressive, lay "outside the user's scope", thereby slightly diluting its focus.

EdTech (Python)

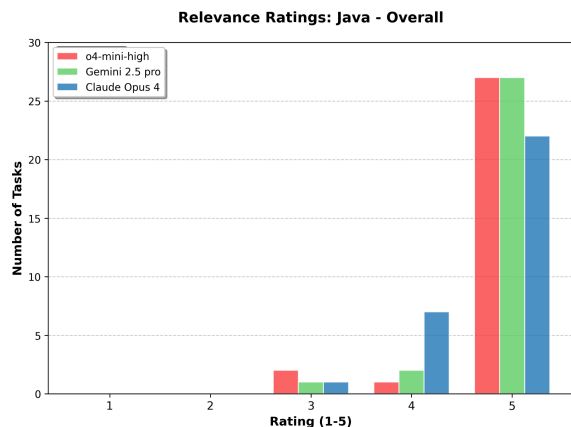


Gemini 2.5 Pro and Claude Opus 4 are the top-performing models in terms of relevance, demonstrating a flawless record across all evaluations. Both models consistently produced solutions that were described as "strictly" and "entirely focused on the Prompt" requirements. In every instance, their output aligned perfectly with the user's request, including all functional components, without introducing any extraneous or irrelevant code. **OpenAI o4-mini-high also performed exceptionally well**, generating highly relevant solutions that directly fulfilled the Prompt's goals in almost all cases. However, it was penalized

for relevance in one instance where it only partially addressed the user's request. While it successfully targeted the primary goal, it failed to deliver a specific and complex functional

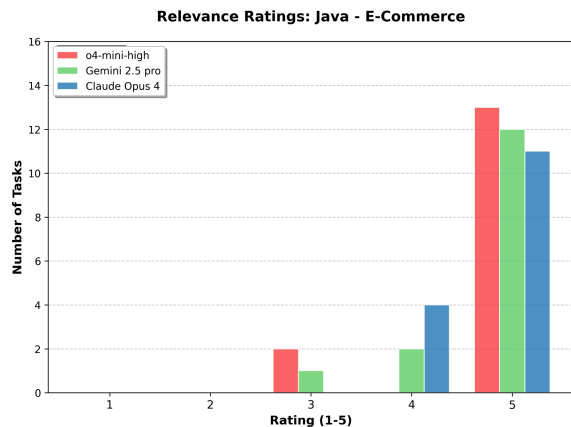
requirement related to "min-cost prerequisite selection," making its solution less relevant than those of its competitors who successfully implemented the entire request.

Java Language



In the Java relevance ratings, **o4-mini-high** and **Gemini 2.5 Pro** are tied with the highest number of perfect 5 ratings, while **Claude Opus 4** trails behind. Given this clear lead in top scores, **OpenAI o4-mini-high and Gemini 2.5 Pro are joint winners for Java**, with Claude performing moderately.

E-Commerce (Java)

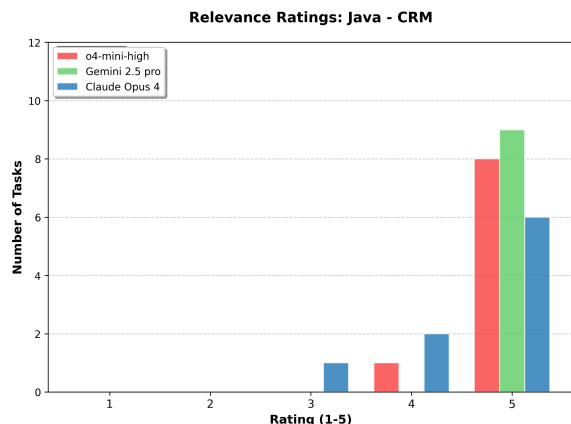


OpenAI o4-mini-high stands out as the leader, securing the most perfect scores. It consistently demonstrates a precise understanding of user requirements, delivering solutions that "Fully meet Prompt" specifications. Justifications frequently highlight its direct alignment with all requested features, from "CSV input/output, predictions, [and] flags" to complex inventory management systems. Its relevance is only diminished when its core explanation of the Provided code becomes "factually incorrect,"

which, while rare, shows a significant deviation from the user's goal of understanding the code.

Gemini 2.5 Pro and Claude Opus 4 are tied as strong competitors, both also achieving numerous perfect scores. **Gemini's** relevance is occasionally reduced when its solutions are "over-engineered" or when key features like "predictive analytics are simulated without a functional random forest" failing to deliver the full functionality requested. **Claude's** relevance suffers when it deviates from specific output requirements, such as when its "output format diverges from what was specifically requested," or when its implementation of key features, like "data Protection compliance is superficial."

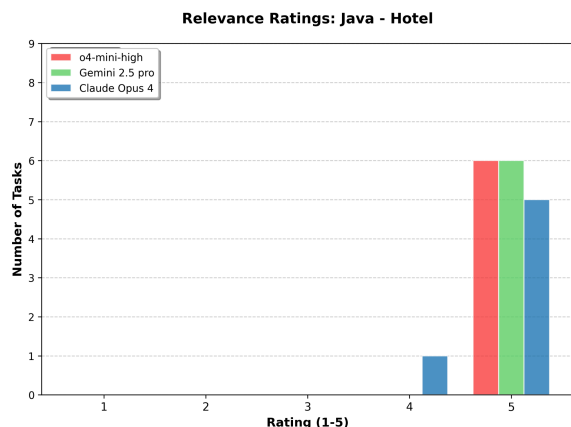
CRM (Java)



Gemini 2.5 Pro is the clear leader, demonstrating flawless alignment with user intent across all Prompts. Its solutions were consistently described as "perfectly relevant" and designed to "directly meet Prompt goals." Justifications repeatedly emphasize that its design "precisely targets the requirements laid out in the Prompt with no extraneous or missing components," highlighting its exceptional reliability and focus.

OpenAI o4-mini-high is a very close second, delivering highly relevant solutions that almost always hit the mark. It received numerous perfect scores, with justifications noting that its responses "directly address the Prompt" and "strictly use core Java and implement rule-based logic aligned with CRM scoring needs." Its relevance was only slightly penalized in one instance where the solution was deemed "less advanced in extensibility and behavioral signal handling" compared to a competitor, indicating that while it fulfills the core request, it may not always capture the more advanced implicit needs.

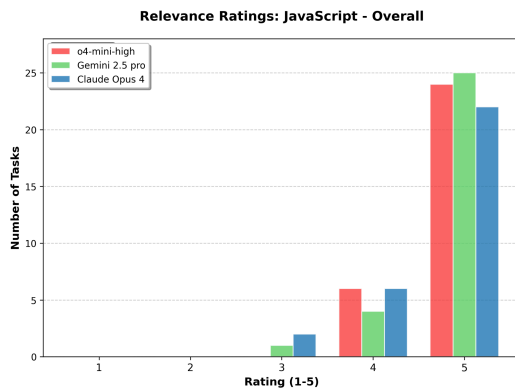
Hotel (Java)



OpenAI o4-mini-high and Gemini 2.5 Pro are the clear leaders, both achieving a perfect score on every single Prompt. They demonstrated an exceptional and identical ability to align their solutions perfectly with the user's intent. Justifications for their performance consistently confirm that they "Fully addressed the Prompt" requirements, delivering solutions that were "exactly matching the Prompt" and "fully aligned with the requirements." Both models successfully incorporated all specified business logic,

technical requirements, and constraints without introducing any extraneous or irrelevant components. **OpenAI o4-mini-high** consistently demonstrates an outstanding ability to interpret and adhere to every detail of the Prompt, resulting in perfectly aligned solutions. Evidence for this includes justifications describing its output as a "Single-file Java tool exactly matching the Prompt" and being "Fully aligned with the Prompt," showcasing its reliability in meeting user needs precisely.

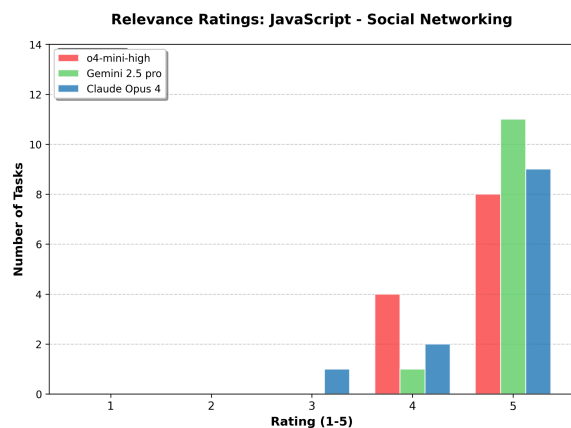
JavaScript Language



Overall

In JavaScript relevance ratings, **Gemini 2.5 Pro** slightly edges out others with the highest number of perfect 5 ratings, while **OpenAI o4-mini-high** closely follows and **Claude Opus 4** trails a bit. Thus, **Gemini 2.5 Pro is the narrow winner for JavaScript**, with strong but nearly matched competition.

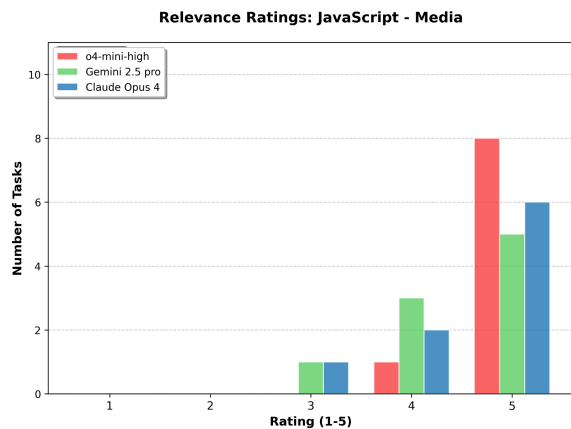
Social Networking (JavaScript)



Gemini 2.5 Pro stands out as the leader, securing the most perfect scores. It consistently demonstrates an exceptional ability to align its solutions with the user's intent. Justifications for its performance repeatedly state that it is "Precisely aligned with all Prompt demands" and "Perfectly addresses all Prompt requirements." This indicates a superior ability to interpret and fulfill every aspect of a complex request without deviation.

OpenAI o4-mini-high consistently and comprehensively addresses the core requirements of a Prompt, making it highly relevant for a wide range of tasks. However, its relevance can be slightly reduced when there is a "slight mismatch" between its chosen architecture and the specific environment described in the Prompt, such as Providing a solution more tailored to web than mobile.

Media (JavaScript)

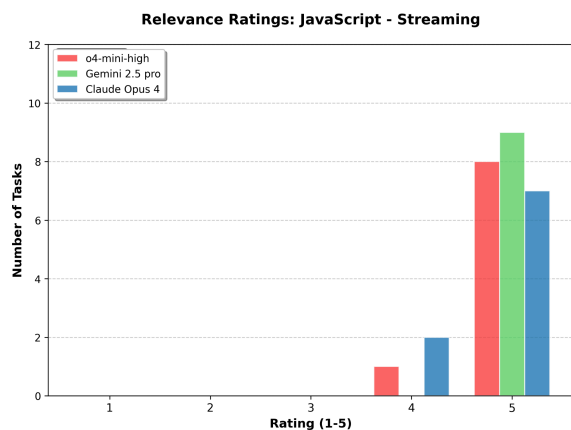


OpenAI o4-mini-high stands out as the leader, securing the most perfect scores. It consistently demonstrates the most precise and direct alignment with the user's goals. Justifications for its numerous perfect scores repeatedly state that it "Directly matches Prompt" requirements and "Directly fulfills all elements of the Prompt with no off-topic content." Its relevance was only slightly reduced in one instance for a minor technical deviation (using `mp4box.js` instead of pure Web APIs), which still successfully addressed the core

Problem.

OpenAI solutions are consistently well-aligned with the user's goals, as it "Directly fulfills all elements of the Prompt with no off-topic content." Its relevance is only slightly reduced when it makes a technical choice, like using a specific library, that deviates from the ideal implementation but still achieves the desired outcome.

Streaming (JavaScript)



Gemini 2.5 Pro is the undisputed leader, securing a perfect score across all Prompts. Its flawless record is supported by justifications that consistently state it addresses "Every major feature of the Prompt directly" and "Fully meets the Prompt's scope." It demonstrates an exceptional ability to interpret and fulfill all complex, multi-layered requirements of streaming engineering without any deviation.

OpenAI o4-mini-high is an extremely close second, also achieving a near-perfect record. It is consistently praised for being "Entirely focused

on the requirements" and for solutions that "Directly target the Prompt's goals." Its only deduction was for a "shallow treatment of Prompt components" in one instance, indicating that while it is highly relevant, it can occasionally lack the depth of the top performer.

Completeness

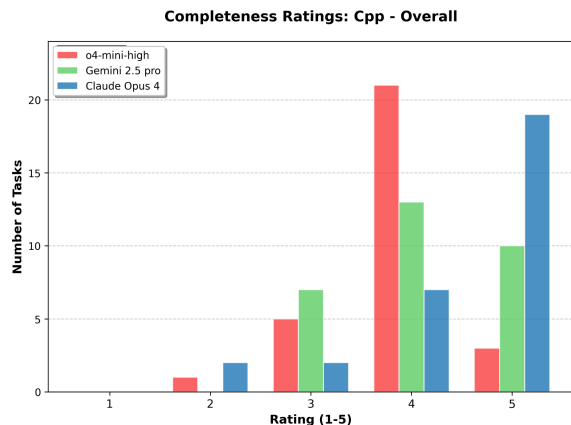
Completeness measures how fully and comprehensively an LLM generates code to meet the requirements of a given task or Prompt. It's a measure of whether the code outputs:

- Covers all aspects and functionalities explicitly or implicitly requested in the Prompt.
- Handles all specified or reasonable edge cases and scenarios.
- Provides a self-contained and ready-to-use solution, including necessary imports, configurations, or setup if applicable.

Completeness Ratings - Cross-Language Domain Comparison



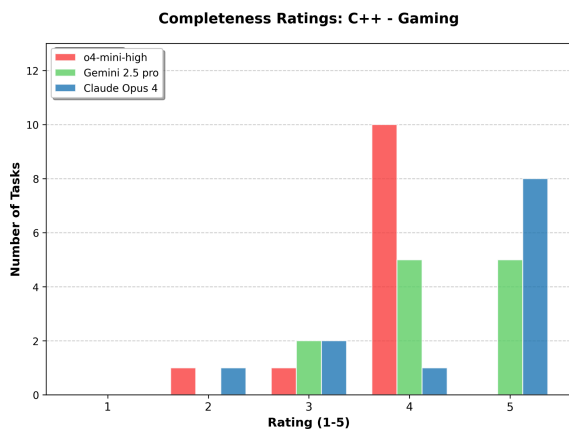
C++ Language



Overall

In the C++ completeness ratings, **Claude Opus 4** dominates with the highest number of perfect 5 scores, while **OpenAI o4-mini-high** peaks at rating 4 and lags significantly in top ratings. Therefore, **Claude Opus 4** is the clear winner in completeness for C++, showing superior thoroughness across tasks

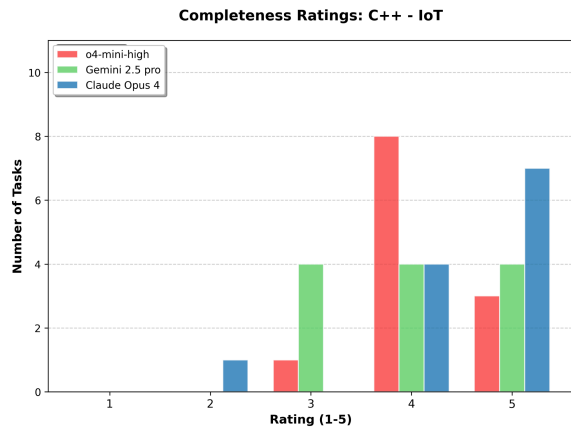
Gaming (C++)



Claude Opus 4 stands out as the leader. Its solutions are consistently praised for being exceptionally thorough. Justifications for its perfect scores frequently describe its output as "very complete" and note that it "covers all required components from the Prompt comprehensively," including complex systems like physics, serialization, pathfinding, and even a "debug renderer." This indicates a strong capability to understand and implement every part of a complex, multifaceted request.

OpenAI o4-mini-high ranks third, as its solutions consistently lack critical components required by the Prompts. It frequently provides code that is only a "skeleton" or is missing key features like "AABB-circle collisions." In the most severe case, its response was judged to have omitted almost every core requirement, including "collision detection, shapes, physics world, visualization, or debug mode," making it fundamentally incomplete.

IoT (C++)

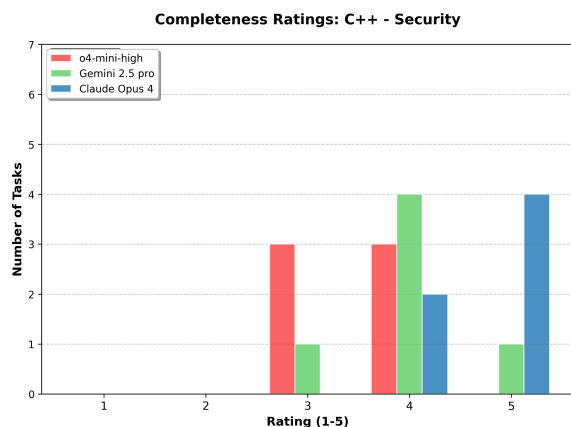


Claude Opus 4 stands out as the leader, securing the most perfect scores. It demonstrates a superior ability to deliver exceptionally comprehensive solutions for complex, multifaceted IoT Problems. For instance, it received perfect scores on the intricate home alarm system and the data-driven WiFi signal mapper Prompts, with justifications praising its solutions for covering all major requirements "extensively" and "with detailed implementation." This indicates a strong capability to Produce

fully-featured, end-to-end systems.

OpenAI o4-mini-high consistently covers the major requirements of a Prompt but frequently lacks the robust, Production-grade features needed for a truly complete solution. Evidence for this includes justifications noting its lack of "configuration validation," "robust error handling for GPIO operations," and providing only "minimal error recovery mechanisms."

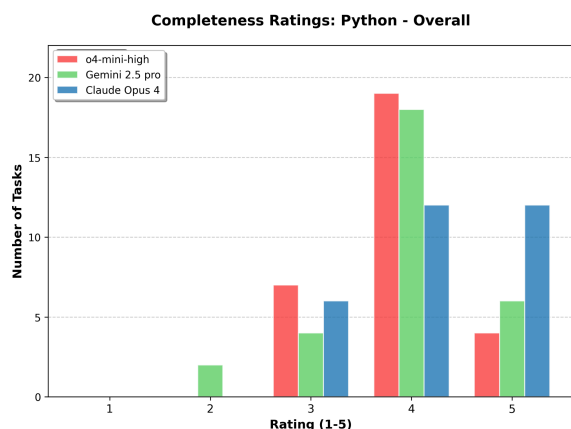
Security (C++)



Claude Opus 4 stands out as the clear leader. It consistently delivers the most comprehensive and Production-ready solutions for complex security Prompts. Justifications for its perfect scores repeatedly state that its responses "cover all Prompt points: robust TLS, latency logging, error handling, HTTP parsing, [and] Production readiness." Even when its solutions were not perfect, the deductions were for minor issues like "limited dictionary size for Production," indicating an already high baseline of completeness.

OpenAI o4-mini-high solutions consistently deliver the basic structure of the request but omit the critical, Production-ready features required for security applications. Evidence for this includes its failure to Provide a "detailed vulnerability reporting structure" and its omission of "robust certificate validation" and "timeout handling," which are non-negotiable in a fintech context.

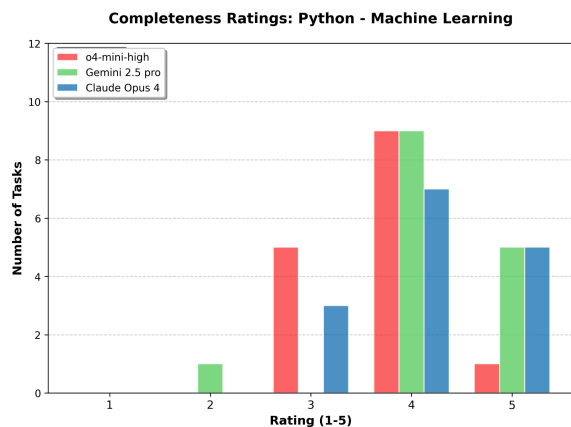
Python Language



Overall

In Python completeness ratings, **Claude Opus 4** secures the highest number of perfect 5 scores, while **OpenAI o4-mini-high** and **Gemini 2.5 Pro** cluster around rating 4. Although **o4-mini-high** leads in 4s, **Claude Opus 4** is the overall winner due to its stronger top-end completeness performance.

Machine learning (Python)



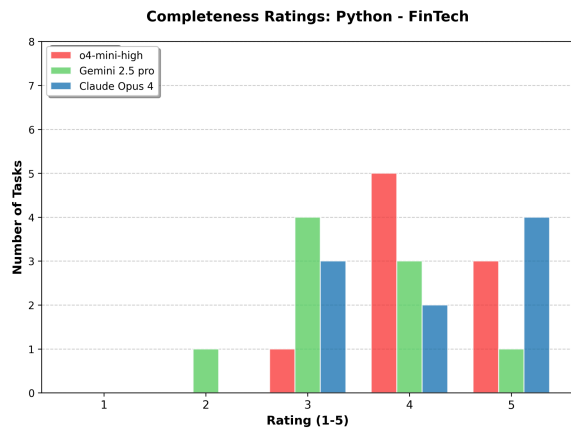
Gemini 2.5 Pro and **Claude Opus 4** are the leading models, achieving an equal number of perfect scores.

Gemini 2.5 Pro consistently delivers highly comprehensive and robust solutions. It earned top marks for an "Extremely comprehensive" neuro-engineering platform that included "realistic hierarchical brain Processing" and a complete robotics system. Its completeness was only marked down when it missed very specific implementation details like "real-time Processing" or "feature attribution scores," while still providing

a nearly end-to-end framework.

OpenAI o4-mini-high's solutions often feel like a blueprint rather than a finished Product, covering the core architecture but omitting key functional and Production-ready features. Evidence for this includes its failure to Provide "real privacy compliance" or an "API/service interface," and leaving critical security elements "as placeholders" for the user to implement.

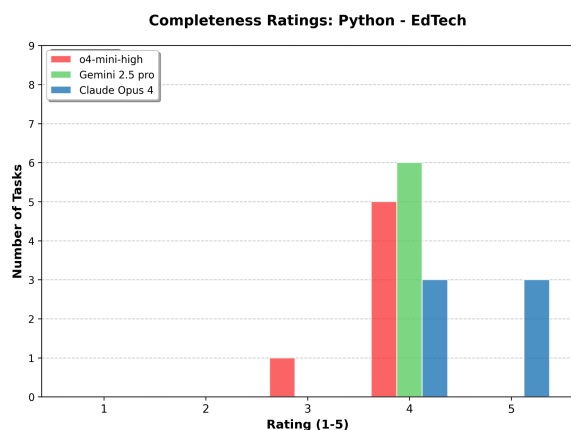
FinTech (Python)



Claude Opus 4 is the leading model. It excels at delivering solutions that are not just complete but often exceed the Prompt's requirements in a Production-ready manner. For instance, justifications for its perfect scores praise it for a "Comprehensive implementation covering all requirements with real API integration" and for delivering a "full ecosystem" that includes a rich set of features like data validation, mock implementations for testing, and a full suite of unit tests. In another case, its solution was lauded for "comprehensively covering all requested features

and more," including advanced features like Greeks computations and input validation. **OpenAI o4-mini-high** is an extremely close competitor, achieving the second-highest number of perfect scores and demonstrating remarkable consistency. Its primary strength lies in precisely fulfilling every explicit requirement of a Prompt. Justifications for its perfect scores note that "All twelve explicit requirements from the Prompt are covered" and "All requested components are present," highlighting its reliability in mapping its output directly to the user's request. Even when not scoring perfectly, its solutions are described as covering "all major requirements comprehensively," indicating a high baseline of completeness. The key difference between the top two models lies in their approach. **Claude Opus 4** is more likely to deliver an all-encompassing, ecosystem-level solution, but can be inconsistent, as evidenced by its lower scores where it missed "critical mechanisms" like dynamic rule updates and rate limiting.

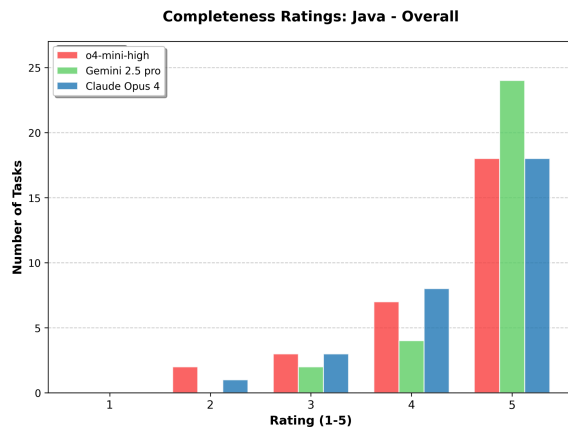
EdTech (Python)



Claude Opus 4 stands out as the leading model for completeness in the EdTech domain. It was the only model to receive perfect scores, with justifications highlighting that it "Fully addresses all Prompt requirements" and enhances its solutions with "robust testing, detailed reporting, fairness considerations, and session management." Even when not scoring perfectly, its solutions were considered strong approximations, indicating a high level of functional completeness. **OpenAI o4-mini-high** ranked third, as its solutions were frequently

described as functionally incomplete Prototypes. Justifications repeatedly noted that its implementations were "skeletal or basic" and lacked "advanced Production features such as comprehensive error handling, input validation, [and] scaling tests." Its most significant failure was in a response where "major logic. is only described or stubbed and not executable," making it the least complete of the three models.

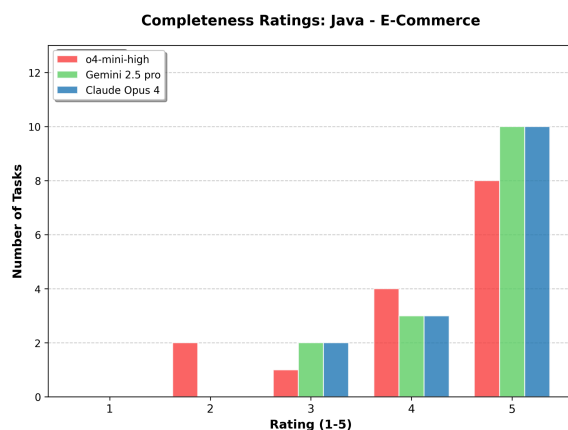
Java Language



Overall

In Java completeness ratings, **Gemini 2.5 Pro** leads clearly with the highest number of perfect 5 scores, outpacing both **Claude Opus 4** and **OpenAI o4-mini-high**. Thus, **Gemini 2.5 Pro** is the definitive winner in Java completeness, showing stronger task coverage and completeness.

E-commerce (Java)

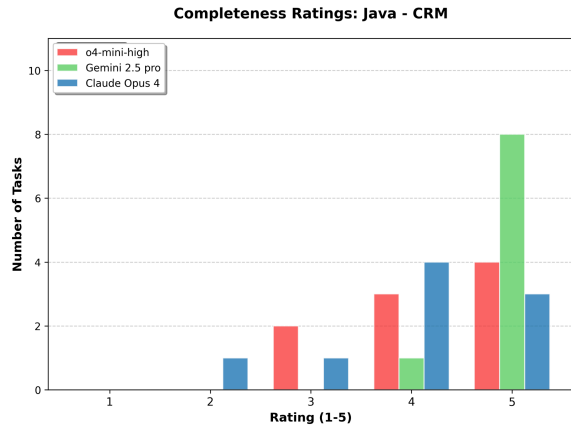


Gemini 2.5 Pro and **Claude Opus 4** are the leading models, achieving an equal number of perfect scores. They both consistently demonstrate a strong ability to deliver comprehensive solutions that cover all specified requirements.

OpenAI o4-mini-high's completeness is inconsistent; it can deliver a fully working solution for a straightforward task but often omits critical Production-level features in more complex ones. For example, it can deliver a solution that "Does everything," but can also have "significant gaps,"

such as missing "Proper logging mechanisms" or "explicit data Protection compliance details," which are essential for real-world e-commerce systems.

CRM (Java)

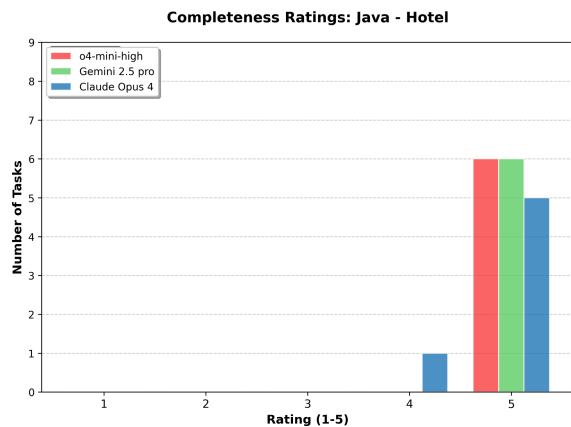


Gemini 2.5 Pro is the undisputed leader, delivering functionally perfect code across all Prompts. Its solutions are not just correct, but robust and well-architected. Justifications consistently praise it for implementing "accurate rule evaluations," maintaining a "score context with justifications," and using an "advanced architecture with domain, application, and rule engine layers." It excels at creating "Production-ready" code that "implements all required features" correctly, from business logic

to critical components like input sanitization and API exposure.

OpenAI o4-mini-high's correctness often reflects a surface-level implementation; the code runs and performs the basic requested task, but it lacks deeper, enterprise-grade considerations. Evidence for this includes its functionally correct scoring logic that fails to Provide auditability by not logging which rules were triggered, and its minimal compliance features that fulfill the letter but not the spirit of the requirement.

Hotel (Java)



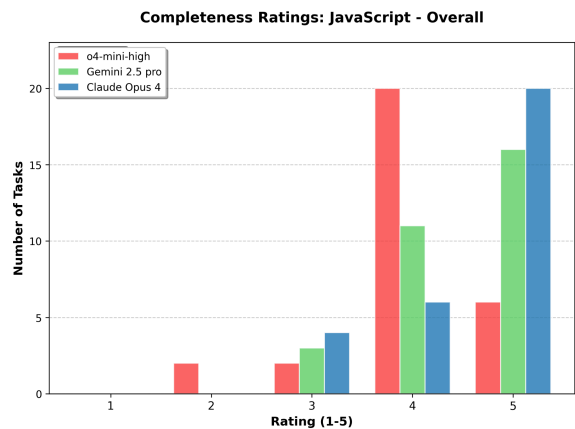
OpenAI o4-mini-high and Gemini 2.5 Pro are the clear leaders, both achieving a perfect score on every single Prompt. They consistently delivered exceptionally complete solutions that addressed all explicit and implicit requirements. Justifications for their performance frequently state that all required components are implemented and every part of the Prompt is explicitly addressed. They reliably covered all required functionalities, including complex business logic like linked/suite handling, security

considerations like PII masking, and technical requirements such as concurrency support and "robust error handling."

OpenAI o4-mini-high consistently demonstrates the ability to deliver solutions that cover all core requirements from end to end, making them feel like finished products. Evidence from the

justifications confirms it successfully "Handles bookings, maintenance, linked rooms, overlap checks, masking, input validation, and error handling" and provides a complete package with "run instructions."

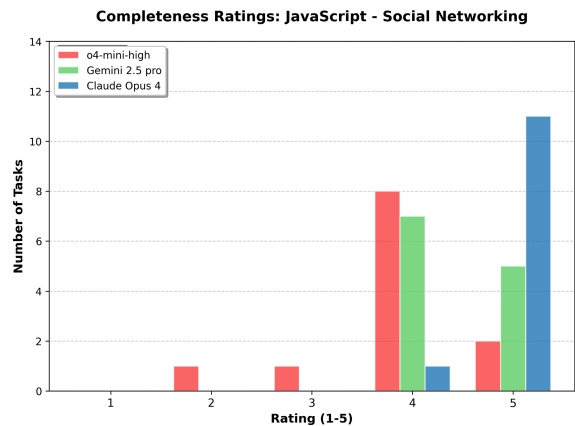
JavaScript Language



Overall

In JavaScript completeness ratings, **Claude Opus 4** leads with the highest number of perfect 5 scores, surpassing **Gemini 2.5 Pro** and **OpenAI o4-mini-high**, which peaks at 4. Despite **OpenAI o4-mini-high** strong 4-ratings, **Claude Opus 4** is the clear winner, delivering more fully complete responses overall.

Social Networking (JavaScript)

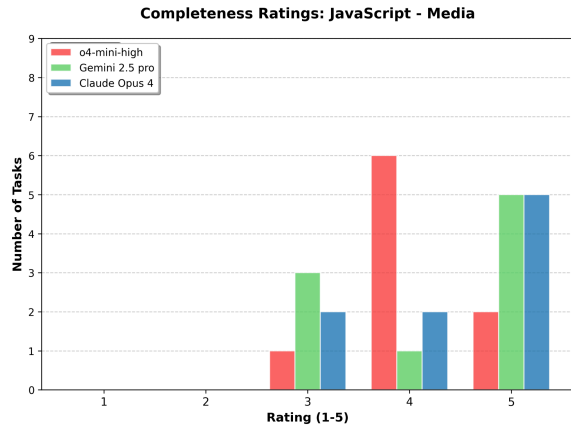


Claude Opus 4 stands out as the clear leader. It consistently delivers exceptionally comprehensive solutions that often go beyond the Prompt's requirements. Justifications for its numerous perfect scores praise it for providing a "Complete module with full TS code," "extensive configuration options," and for covering "all specified requirements and [going] beyond expectations" with features like "typing indicators, user online/offline status, [and] message history." This demonstrates a superior ability to Produce

fully-featured, Production-ready components.

OpenAI o4-mini-high consistently delivers functional solutions that cover the main requirements of the Prompt but often lacks the depth needed for a Production-ready application. Evidence for this includes solutions that are missing critical features like "message persistence or reconnection logic" for a chat system and "auto-save functionality" for an offline draft component, making them incomplete for real-world use.

Media (JavaScript)



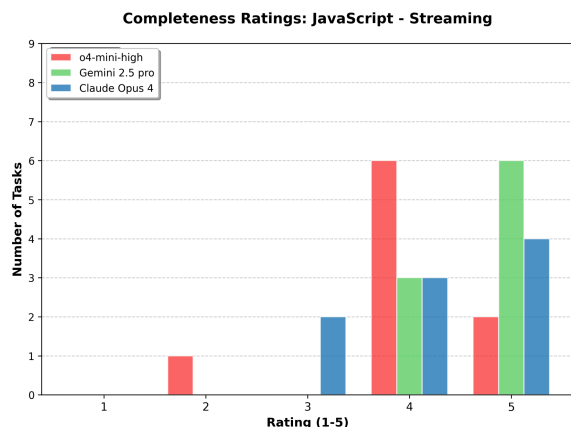
Gemini 2.5 Pro and Claude Opus 4 are the leading models, achieving an equal number of perfect scores. They both consistently deliver exceptionally comprehensive and feature-rich solutions for complex media-focused Prompts.

OpenAI o4-mini-high ranks third, as it consistently delivers solutions that are less complete than its competitors. While it is capable of achieving a perfect score by covering "all instructions including edge cases and extensibility," it is more frequently penalized for

missing key Production-level features. Justifications repeatedly point out that its solutions lack "memory optimizations and Progress feedback for large files," have "tests that are commented rather than integrated," and miss advanced requirements like "shader effects."

OpenAI o4-mini-high solutions often cover the core requirements but consistently omit the finer details needed for a robust, Production-ready application. For instance, it provides the main features for a video cutter but is missing "memory optimizations and Progress feedback for large files," and its visualizer lacks the requested "shader effects."

Streaming (Javascript)



Gemini 2.5 Pro stands out as the leader, securing the most perfect scores. It consistently delivers comprehensive solutions that cover all core requirements and often include advanced features. Justifications praise its solutions for being highly complete, covering "all required functionality, regulatory constraints, test hooks, and config samples," and for including "a modular backend abstraction, Profanity filtering, [and] admin controls." This indicates a strong ability to Produce robust, feature-rich modules suitable for Production streaming platforms.

OpenAI o4-mini-high's solutions provide a functional starting point but consistently lack the comprehensive features and robustness required for modern streaming platforms. Evidence for this includes responses where "Major components like evaluation, error handling, integration are

missing" and where critical details like "resumable playback after interruptions" and "detailed error handling for MSE failures" are omitted.

Correctness

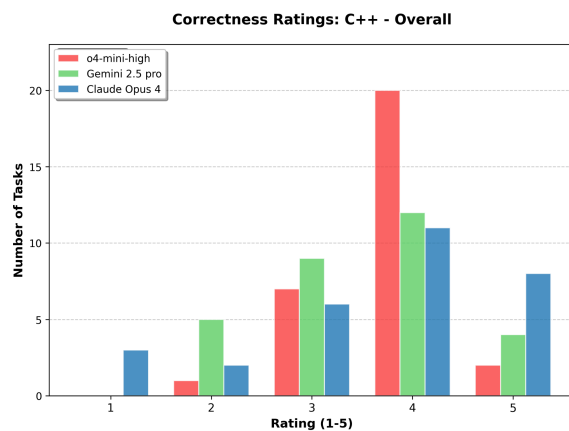
Correctness measures how *fully and correctly* an LLM generates code to meet the requirements of a given task or Prompt. It's a measure of whether the code outputs:

1. **Implements the intended functionality** as described in the Prompt.
2. **Includes all necessary components** (e.g., function definitions, imports, return statements).
3. **It is syntactically and semantically valid**, and can run without critical errors (possibly with minor fixes).

Correctness Ratings - Cross-Language Domain Comparison



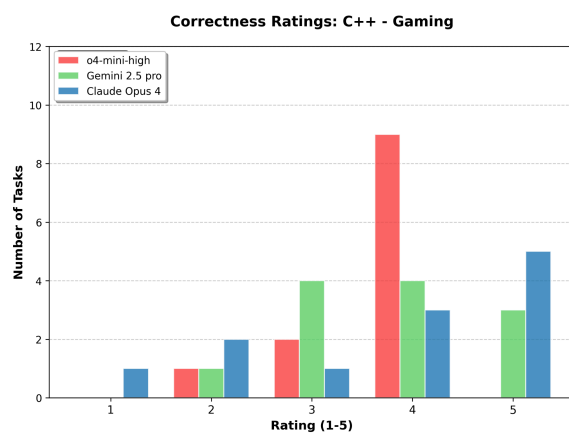
C++ Language



Overall

In C++ correctness ratings, **Claude Opus 4** leads with the highest number of perfect 5 scores, while **OpenAI o4-mini-high** peaks at rating 4 but lags at the top. Despite **Gemini's** balanced spread, **Claude Opus 4** is the winner, delivering the most consistently correct C++ responses at the highest standard.

Gaming (C++)

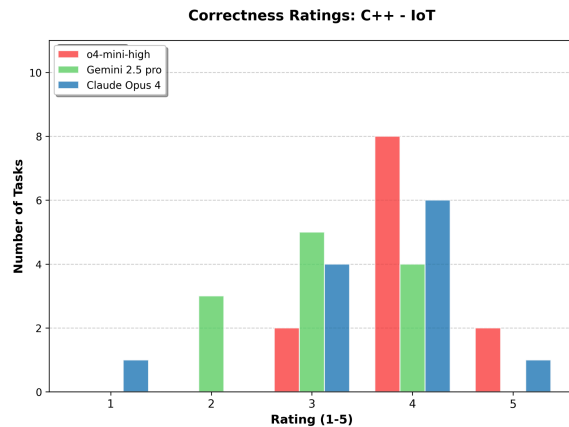


Claude Opus 4 stands out as the leader. It demonstrates the ability to deliver exceptionally correct and robust code for complex gaming systems. Justifications for its perfect scores praise its "fully correct physics formulas, collision detection, friction, restitution, mass/inertia, and positional correction" and describe its Procedural generator as "fully accurate, Production-ready, logically structured code" with "no clear errors." However, its performance is highly volatile; it is also prone to catastrophic failures, receiving the lowest scores on the PBR Prompt where its

"fundamental infrastructure... is non-functional," rendering the entire application useless.

OpenAI o4-mini-high code often appears functional on the surface, but contains underlying correctness issues that would prevent Production use. For example, its physics system had correct basic integration but omitted critical components like "Proper mass/inertia handling" and introduced "memory leaks," making the solution technically unsound.

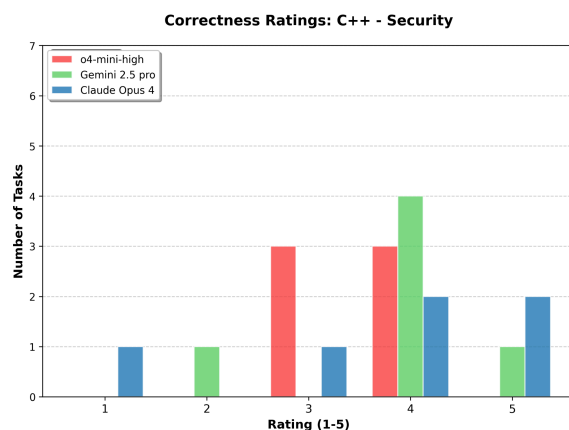
IoT (C++)



OpenAI o4-mini-high is the leader. It is the only model to achieve perfect scores, demonstrating its ability to Produce logically sound and syntactically correct code for complex IoT simulators. However, its correctness is not always consistent. It is frequently penalized for subtle but important Production-readiness issues, such as using non-thread-safe functions like `std::gmtime()`, having fragile command-line parsing logic, and including code with potential race conditions.

OpenAI o4-mini-high code is often logically sound at a high level, but can be undermined by a lack of robustness in its implementation details. For example, its GPIO logic "doesn't check if the pin is already exported, potentially causing errors on restart," a subtle flaw that would compromise the reliability of a real-world system.

Security (C++)



Claude Opus 4 is the leader, securing the most perfect scores. It demonstrates the ability to deliver exceptionally correct and Production-ready code for complex security tasks. For instance, its TLS client was described as "fully correct, compiled and logically consistent" and correctly handled advanced non-blocking socket logic. However, its performance is extremely volatile, as it also received the lowest possible score for a password estimator that had "Several compilation errors," "incorrect syntax," and "logical flaws,"

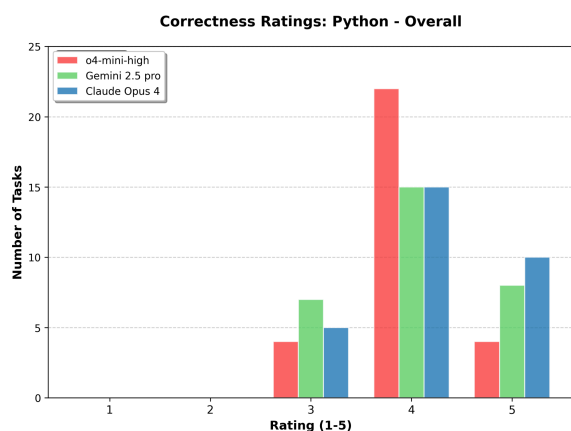
rendering it completely non-functional.

Gemini 2.5 Pro ranks second. While it earned a perfect score for its sound entropy calculation logic, it also Produced code with severe correctness issues. It was heavily penalized for a password estimator that would have "Several compilation errors," including incorrect use of C++23 features and "potential bounds checking issues." In another case, its timeout

implementation was flagged as "not thread-safe" and likely to "cause race conditions in high-scale implementations."

OpenAI o4-mini-high ranks third, failing to achieve a single perfect score. Its code consistently suffers from conceptual and logical flaws, even if it avoids the compilation errors seen in other models. It was marked down for having a "flawed" entropy calculation model that used an inaccurate method of subtracting penalties. Furthermore, its code was described as having "logical gaps" and "likely wouldn't run as given," indicating that while it might be syntactically closer to correct, it is functionally unsound.

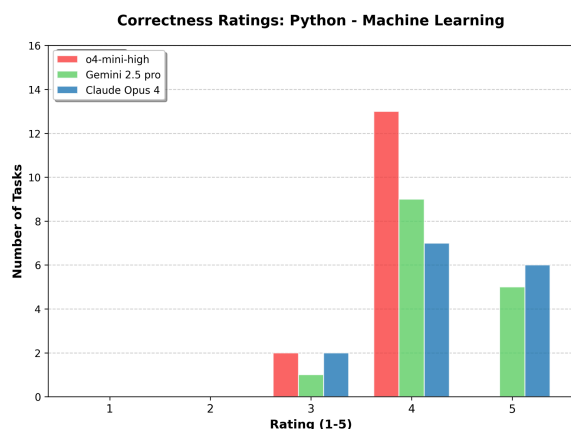
Python Language



Overall

In Python correctness ratings, **Claude Opus 4** leads with the highest number of perfect 5s, while **OpenAI o4-mini-high** dominates at rating 4 but trails at the top end. Thus, **Claude Opus 4** emerges as the winner, offering the most accurate solutions overall despite **o4-mini-high**'s strong showing at near-perfect levels.

Machine Learning (Python)



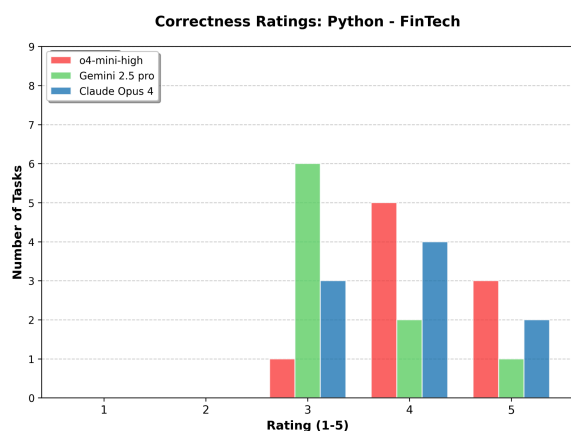
Claude Opus 4 stands out as the leading model, securing the most perfect scores. It demonstrates the ability to produce highly correct and robust code for complex machine learning tasks. For instance, it received top marks for its sound implementation of various model architectures (VAE, Transformer, GNN) and for its effective PII redaction capabilities. However, its performance can be inconsistent; justifications also point to significant flaws, such as code that "Execution fails due to invalid input handling" and a placeholder model that "lacks any statistical, ML-based, or signal decomposition logic,"

indicating a high variance between exceptional correctness and critical failure.

OpenAI o4-mini-high's correctness issues often stem from a surface-level implementation of advanced concepts, leading to code that runs but misses the Problem's core technical

requirements. Evidence for this includes a "Prototypical network" implementation [that] is overly simplistic" and PII redaction patterns that "would miss many real-world cases," highlighting a recurring pattern of fundamental technical shortcomings.

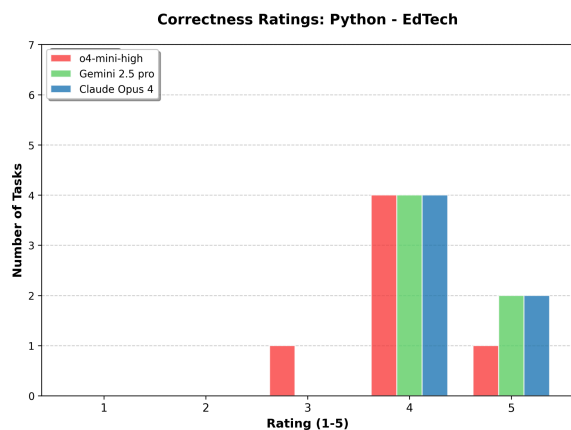
Fintech (Python)



OpenAI o4-mini-high achieves the highest average score, establishing it as the leading model in this dataset. It excels at Producing code that is technically sound and correctly implements the core requirements of a Prompt. For instance, in the Monte Carlo simulation task, it received two perfect scores for its mathematically sound logic and accurate payoff calculations. Similarly, for the fraud detection framework, it was commended for correctly implementing the required rule-based and machine learning detection mechanisms. OpenAI

o4-mini-high's primary weakness, despite its high scores, is a tendency to omit peripheral, yet critical, enterprise-grade features. A key example is in the complex FinTech microservice Prompt, where it successfully built the core rebalancing logic but failed to include the specified HMAC authentication or Proper TLS certificate handling, which are essential for a Production environment. In contrast, **Claude Opus 4**, while not the highest scorer on average, demonstrated exceptional capability on the most complex Prompt, where it earned perfect scores for delivering a solution with "perfect technical accuracy," including the advanced security and asynchronous features that the other models missed. Its lower overall score is due to weaker performance on the other Prompts, where it was penalized for issues like flawed calculations or incomplete implementations. **Gemini 2.5 Pro** scored the lowest of the three, with justifications frequently pointing to significant deviations from the Prompt's requirements, such as using mock API calls instead of real integration or implementing only a single, narrow variant of a required feature.

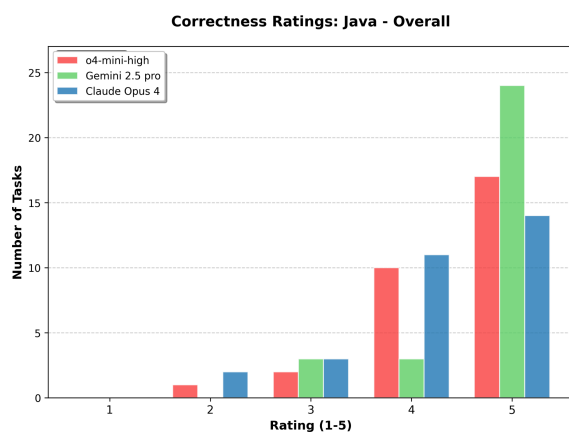
EdTech (Python)



Gemini 2.5 Pro and Claude Opus 4 are the top-performing models, achieving the same average score in the EdTech domain. They both consistently deliver code that is functionally correct and technically sound, though they exhibit different strengths and weaknesses. **Gemini 2.5**

Pro excels at implementing the core requested features correctly. Justifications praise its use of a "realistic bidirectional transformer and Bayesian mastery model." Its lower scores stemmed from minor issues, such as a "slight deviation" on the specific type of encryption requested or having logic that was "partially defined" rather than fully implemented, but it avoided major functional errors. **OpenAI o4-mini-high** ranks third, exhibiting more significant correctness issues. Its most notable failure was a response where a core component, the "min-cost flow part" was "not implemented at all." Other justifications repeatedly described its implementations as "minimal" or "simplistic" and lacking "real-world pretraining" or validation for "cryptographic correctness," indicating a lower level of robustness and correctness compared to the other two models.

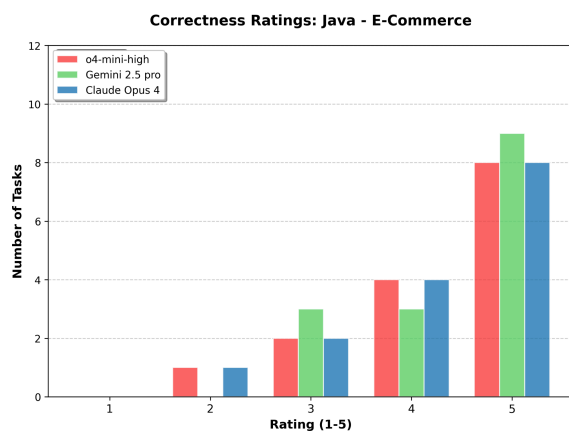
Java Language



Overall

In Java correctness ratings, **Gemini 2.5 Pro** stands out with a dominant number of perfect 5 scores, clearly ahead of **OpenAI o4-mini-high** and **Claude Opus 4**. This makes **Gemini 2.5 Pro** the clear winner, offering the most accurate and reliable Java responses.

E-Commerce (Java)



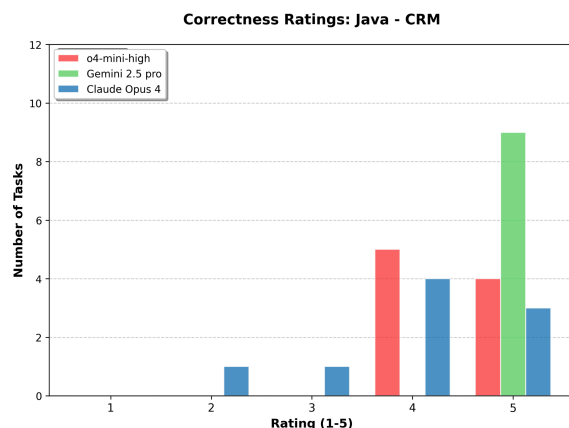
Gemini 2.5 Pro stands out as the leader, achieving the most perfect scores. It consistently delivers correct and functional code across a range of e-commerce Prompts. Its correctness issues, when they occur, tend to be related to "minor issues with assumptions," insecure practices like a "hardcoded" password, or delivering solutions where key components are incomplete placeholders (e.g., "PredictionService lacks a real random forest"). This indicates a high degree of baseline correctness that is occasionally undermined by a lack of

implementation depth or security hardening.

OpenAI 4-mini-high is a strong competitor, also earning numerous perfect scores. It excels at implementing the core algorithm of a Prompt correctly, earning praise for "Correct average/std deviation logic" and for code that is "syntactically correct and logically sound." However, its correctness is often brittle and lacks Production robustness. It is frequently penalized for "significant robustness issue's," such as using Integer.parseInt() "without any exception handling," which would cause the entire Program to crash, and for Providing explanations of its own code that are "factually incorrect."

OpenAI o4-mini-high often implements the central logic correctly, but fails on the robust implementation details required for Production systems. Evidence for this includes its code that crashes from a simple NumberFormatException due to missing exception handling and its explanations that can be "factually incorrect," revealing a critical gap between theoretical correctness and practical reliability.

CRM (Java)



Gemini 2.5 Pro is the undisputed leader, delivering functionally perfect code across all Prompts. Its solutions are not just correct, but robust and well-architected. Justifications consistently praise it for implementing "accurate rule evaluations," maintaining a "score context with justifications," and using an "advanced architecture with domain, application, and rule engine layers." It excels at creating "Production-ready" code that "implements all required features" correctly, from business logic to critical components like input sanitization and

API exposure.

OpenAI o4-mini-high is the second-ranked model, producing code that is generally correct and functional but often lacks the depth and robustness of a Production-grade system. It received perfect scores for implementing a "working lead scoring system" and for solutions where it "correctly handles all functional requirements." However, its correctness was frequently penalized for superficial implementations, such as a "compliance implementation [that] is minimal (just logging) and lacks real policy enforcement" or scoring logic that works but "does not log which rules were triggered."

OpenAI o4-mini-high's correctness often reflects a surface-level implementation; the code runs and performs the basic requested task, but it lacks deeper, enterprise-grade considerations. Evidence for this includes its functionally correct scoring logic that fails to Provide auditability by not logging which rules were triggered, and its minimal compliance features that fulfill the letter but not the spirit of the requirement.

Hotel (Java)

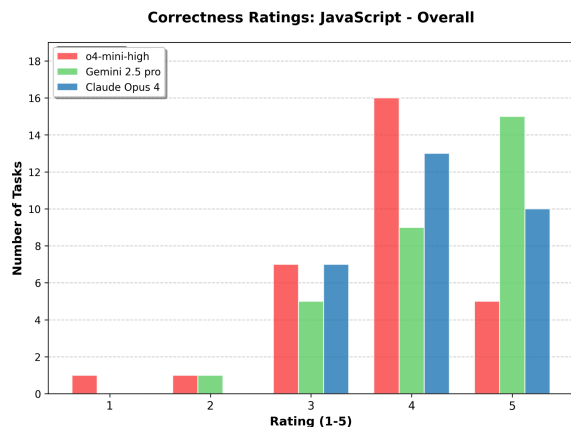


Gemini 2.5 Pro is the undisputed leader, delivering functionally perfect and logically sound code across all Prompts. Its flawless record is supported by justifications praising it for "correctly" handling all business logic, including "bookings, maintenance, and linked rooms with conflict checks." Furthermore, it demonstrates technical excellence by implementing "all pricing logic accurately, including concurrency and seasonal adjustments," and ensuring security requirements like PCI DSS compliance are "observed."

OpenAI o4-mini-high is a very strong second, achieving a near-perfect record. Its code is consistently described as "functionally correct" and "logically sound," correctly handling everything from data parsing and validation to PII masking and complex pricing logic. Its only point deduction was for a subtle issue where the implementation of "real-time" was less continuous than a true Production system might require, a nuance that does not affect the code's immediate correctness but points to a slight gap in conceptual depth.

OpenAI o4-mini-high consistently produces code that is functionally and logically correct, but can sometimes miss the deeper, more nuanced aspects of a Production requirement. For instance, its pricing engine was correct in its calculations but was penalized for not implementing a "true real-time continuous" system, showcasing a gap between a correct implementation and a conceptually perfect one.

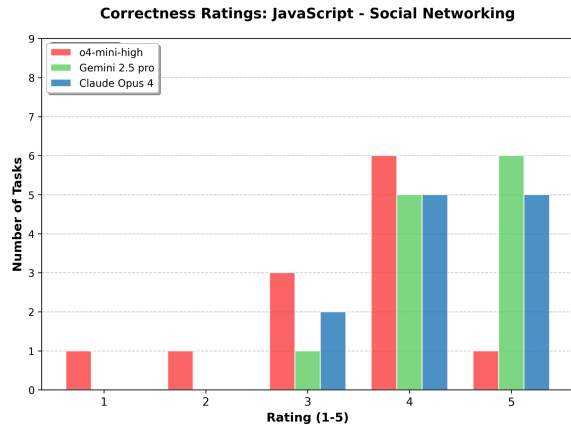
JavaScript Language



Overall

In JavaScript correctness ratings, **Gemini 2.5 Pro** leads with the highest number of perfect 5 scores, while **OpenAI o4-mini-high** shows strength at rating 4 but lags at the top. Hence, **Gemini 2.5 Pro** is the overall winner, demonstrating superior correctness in JavaScript outputs.

Social Networking (JavaScript)



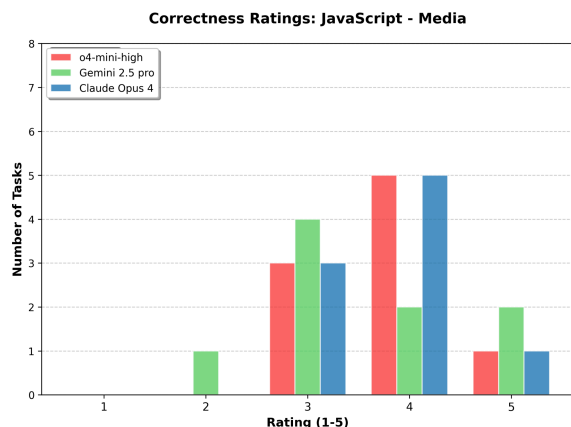
Gemini 2.5 Pro and **Claude Opus 4** are the leading models, achieving an equal number of perfect scores. They both consistently deliver code that is technically sound and logically correct.

OpenAI o4-mini-high ranks third, receiving only one perfect score and consistently exhibiting more severe correctness issues. Its solutions frequently contain "critical flaws," such as an "Automerge P2P sync implementation" with imProper API usage. Other issues include a lack of "Proper exception handling" and implementing

logic, like a global message counter, that "could potentially create race conditions," indicating a pattern of Producing code that is not robust enough for Production use.

OpenAI's solutions often have significant technical issues that undermine their core functionality and reliability. For instance, its P2P sync implementation had "critical flaws," and its chat system's message counter "could potentially create race conditions," making the solutions unsuitable for a real-world social application.

Media (JavaScript)



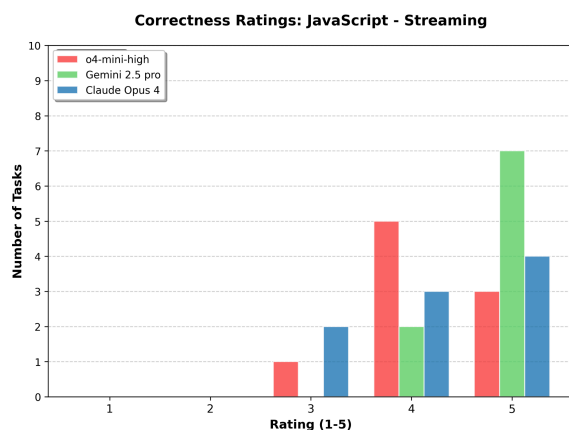
Gemini 2.5 Pro is the leader, securing the most perfect scores. It demonstrates the ability to deliver "Fully accurate implementation[s] with thorough error handling and diagnostic outputs." However, its performance is highly inconsistent. While capable of perfect code, it is also responsible for the most severe failures, producing a video cutter where the "core trimming logic is completely broken" and another solution that "breaks the 'single file' requirement," indicating a high-risk, high-reward Profile.

OpenAI o4-mini-high ranks third, also earning a perfect score but having the lowest average correctness. Its solutions are frequently undermined by fundamental flaws in logic and implementation. For example, its video cutter's "trimming logic is flawed and won't yield a valid video," and it often fails to handle edge cases, such as "corrupted or unsupported files." This

pattern indicates that while it can be correct on simpler tasks, it struggles with the core technical challenges of more complex media Processing.

OpenAI's correctness is often superficial; it produces code that appears functional at a high level but contains fundamental logical flaws that prevent it from working as intended. For example, its video cutter's "trimming logic is flawed and won't yield a valid video," and its audio visualizer "lacks AudioContext.resume() and stream cleanup," both critical failures for Production use.

Streaming (Javascript)



Based on an analysis of the correctness ratings, **Gemini 2.5 Pro** stands out as the leader, securing the most perfect scores. It consistently delivers code that is not only functionally correct but also architecturally sound and Production-ready. Justifications for its perfect scores describe its solutions as a "Fully correct implementation matching TSX spec" and note that "All technical implementations (MSE, RxJS, CDN failover, checksum) are valid and Production-sound." This demonstrates a superior ability to produce reliable, high-quality code for

complex streaming applications.

OpenAI o4-mini-high ranks third among its peers, primarily due to consistently producing code with significant correctness flaws and receiving the fewest perfect scores. While its solutions often contain "technically valid parts," they frequently lack complete evaluation or have notable implementation gaps, such as missing "real segment data" or placeholder functions (e.g., for checksums). This indicates that **OpenAI o4-mini-high** tends to generate simplified, rather than functionally complete or robust, versions of requested tools. Its high-level logic is often sound, but the lack of robustness and completeness in implementation details, like the absence of real segment data for ABR player logic or a placeholder checksum function, ultimately undermines its practical utility.

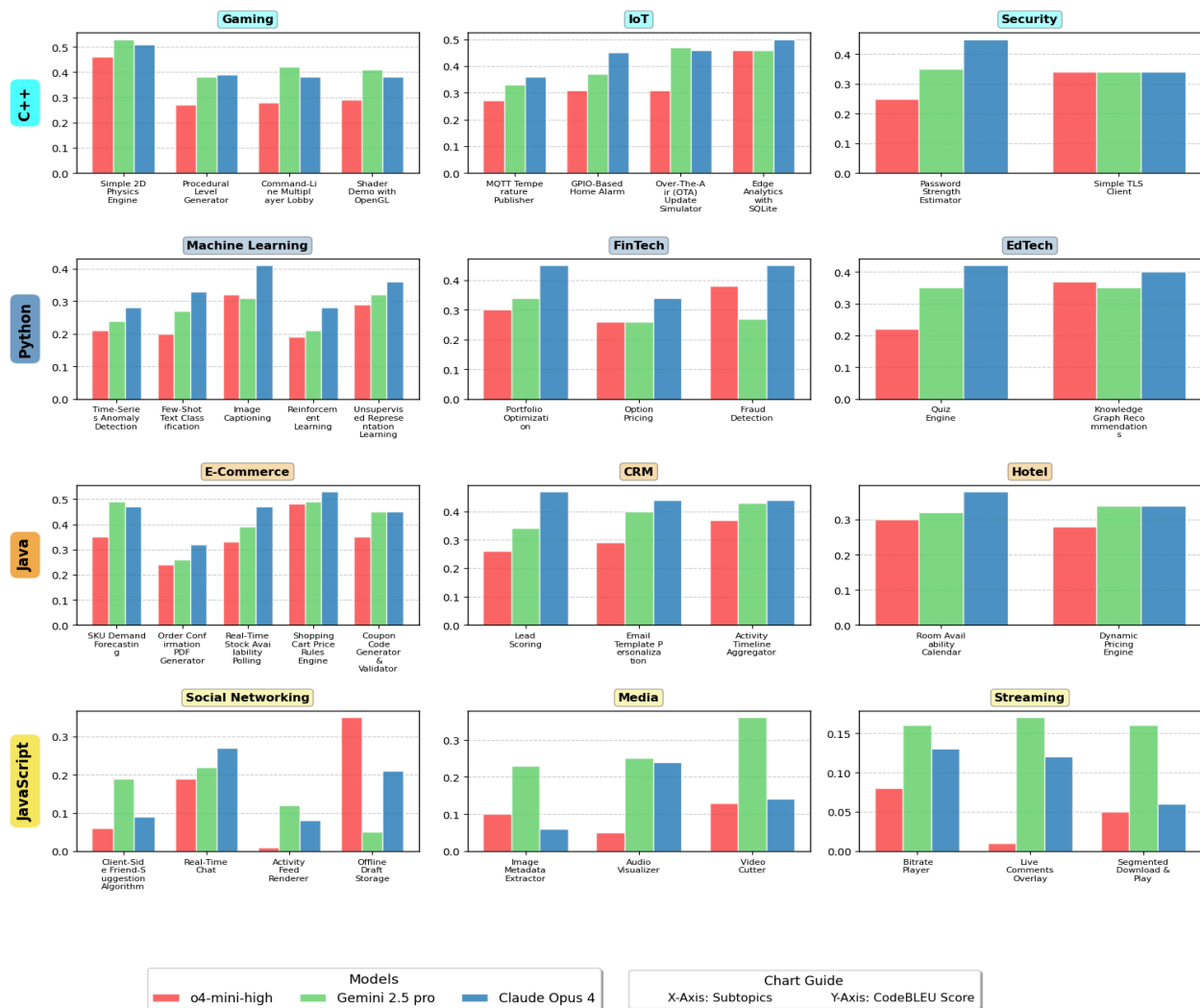
CodeBLEU

CodeBLEU is a composite metric designed specifically for code generation tasks. It evaluates the similarity between the generated code and the reference (golden) code using multiple dimensions:

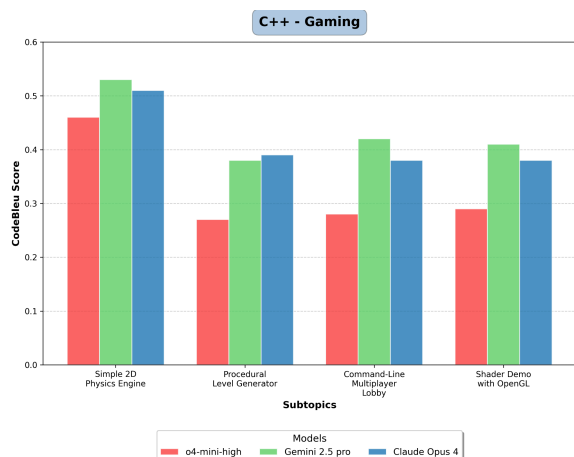
$$\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{BLEU}_{\text{weight}} + \gamma \cdot \text{MatchAST} + \delta \cdot \text{MatchDF}$$

- **BLEU (N-gram match):** Calculated using the standard BLEU metric as defined by research article [4]. It evaluates surface-level n-gram overlap between the candidate and reference code.
- **BLEUweight (Weighted N-gram match):** Compares the candidate code and the reference code tokens with different weights. This helps to emphasize certain n-gram matches more than others.
- **MatchAST (Syntactic AST match):** Measures the syntactic similarity between the candidate and the reference by comparing their Abstract Syntax Trees (ASTs), capturing the grammatical structure of code.
- **MatchDF (Semantic Dataflow match):** Considers the semantic similarity between the Candidate and the reference code by comparing their data flow, how information is passed and transformed.

System Evals - Cross-Language Domain Comparison



Gaming (C++)

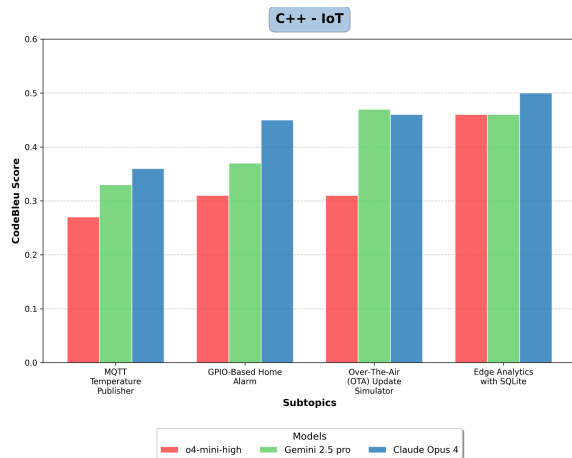


Gemini 2.5 Pro and **Claude Opus 4** consistently produced more sophisticated, Professional, and feature-complete solutions, often demonstrating exceptional expertise in game engine architecture and modern C++ practices. This is reflected in their generally higher **CodeBLEU** scores. **OpenAI o4-mini-high** Provided solid foundational code that was often correct in its core implementation but frequently lacked the completeness, robustness, or modern features required for Production use, resulting in lower

scores and justifications highlighting missing components or errors.

- **OpenAI o4-mini-high** consistently Provided solutions that served as a solid foundation or a correct minimal snippet, but regularly failed to deliver a complete, Production-ready application. Justifications frequently mention that it was mostly incomplete, had critical syntax errors, or was missing key features like collision detection or proper texture mapping. While its output could be a helpful starting point, it consistently required significant additional work to be functional.
- The **CodeBLEU** scores consistently place it as the weakest performer, aligning perfectly with the justifications.
- In the **Level Generator Prompt** (381), it had the lowest score of 0.27. This directly reflects the justification mentioning critical syntax errors that severely impact its immediate usability.
- Its score of 0.28 on the **high-performance server Prompt** (382) was also the lowest, which contrasts with a surprisingly positive justification. This discrepancy suggests that while the code might have read well, its underlying logic and structure (as measured by Dataflow Match) were not as robust as the alternatives.
- For the **PBR Prompt** (383), its lowest score of 0.29 supports the justification noting a compilation barrier and incomplete normal mapping, which limited its practical use.

IoT (C++)



The performance of the models on the C++ IoT Prompts was highly variable and task-dependent, with no single model consistently outperforming the others. **Claude Opus 4** and **Gemini 2.5 Pro** often attempted more sophisticated, feature-rich, and architecturally complex solutions. This ambition was a double-edged sword: when successful, their code was technically advanced, but it also frequently led to critical bugs,

compilation errors, or violations of Prompt constraints. **OpenAI o4-mini-high** generally Produced more straightforward and direct implementations. This often made its solutions more reliable and better at adhering to specific constraints, though they sometimes lacked the depth and Production-level features of the others and could also contain significant flaws.

OpenAI o4-mini-high solutions were generally the most straightforward and direct. It was praised for Providing a technically sound implementation and a bug-free C++ simulator that was fully compilable. This simple approach often made it the most reliable for following constraints. However, it was also criticized for having significant technical flaws (thread safety, overflow risks), missing features, and providing a basic implementation that lacked sophistication.

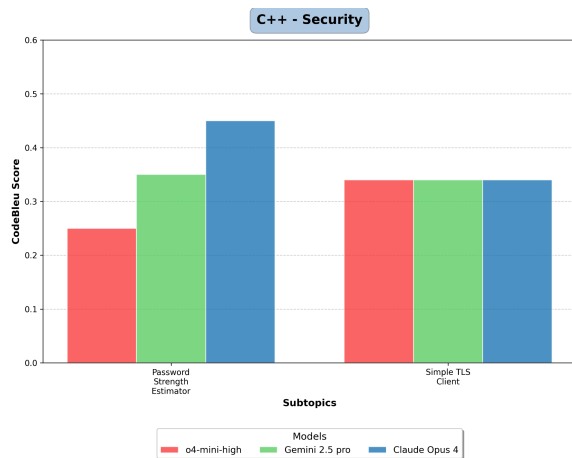
For the MQTT Simulator Prompt (384), it had the lowest score of 0.27. This is a major contradiction with the justification, which called it technically sound and the most practical for deployment because it was the only one to correctly follow all constraints. The low score likely resulted from its simple implementation differing from more complex reference solutions.

In the Alarm System Prompt (385), its low score of 0.31 aligns with the justification that it was a good foundation but lacked polish and advanced features.

For the OTA Simulator (386), its score of 0.31 reflects the justification noting a critical syntax error that prevented compilation.

For the WiFi Mapping Prompt (387), it tied for the highest score with 0.46. This contradicts the justification calling it a basic approach suitable for smaller deployments, suggesting its straightforward implementation was a near-perfect match for the reference solution's structure.

Security (C++)



Claude Opus 4 consistently Produced the most sophisticated, feature-rich, and Production-ready solutions, which was reflected in its high scores and glowing justifications. Gemini 2.5 Pro also delivered high-quality, enterprise-grade code but was sometimes held back by significant implementation flaws or correctness issues.

OpenAI o4-mini-high Provided functional implementations that met the core requirements

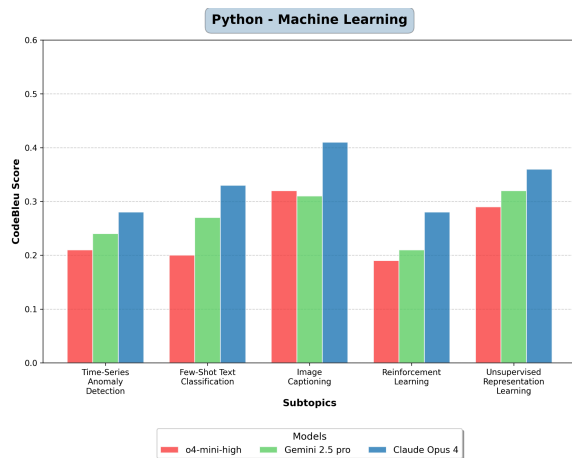
but generally lacked the architectural depth, security robustness, and modern features of the other two models, positioning it as a more basic but practical option.

OpenAI o4-mini-high consistently Provided functional and correct solutions that were well-executed but less advanced. For the password estimator, it offered a functional, basic implementation that was practical but lacked sophistication. For the TLS client, it produced a well-executed, Production-ready implementation with strong fundamentals, though it had "minor technical inefficiencies and limited use of advanced C++23 features, preventing it from achieving a top score.

In the **Password Estimator Prompt (388)**, it had the lowest score of 0.25. This aligns perfectly with the justification describing it as a functional, basic implementation that traded advanced features for simplicity.

For the TLS Client Prompt (389), it also tied for the highest score with 0.34. This is another significant contradiction, as the justification clearly ranks it below the other two, noting it had minor technical inefficiencies and missed opportunities for optimization. The high score indicates its straightforward, structured approach was a very close match to the reference solution's logic flow, even if it was less architecturally advanced.

Machine Learning (Python)



Claude Opus 4 consistently demonstrated superior performance, delivering solutions that were not only functionally correct but also architecturally advanced and Production-ready. This is reflected in its leading **CodeBLEU** scores across nearly all Prompts. **Gemini 2.5 Pro** proved to be a reliable and strong performer, often producing technically correct and well-structured code, though it sometimes missed

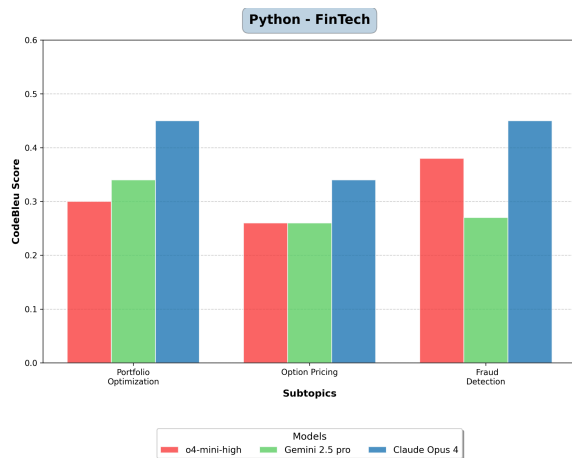
higher-level objectives or advanced features. **OpenAI o4-mini-high** typically managed to create a valid structural baseline but consistently failed to implement the core, complex algorithms required by the Prompts, positioning it as a tool for Prototyping rather than for generating complete solutions.

The justifications consistently describe **Open AI o4-mini-high's** responses as partial, Providing a sound structural baseline or a functional Prototype, but failing on the most critical parts of the Prompts. It regularly "falls short on the deeper technical expectations and lacks the core framework, making its code a starting point that requires significant work to be functional.

Across the Anomaly Detection (360), Legal Tech (361), and Robotics (363) Prompts, **OpenAI o4-mini-high's** Dataflow Match Score was 0.0. This is a critical finding. It quantitatively proves the justifications: the model created code with acceptable syntax (Syntax Match scores are ~0.50), but the fundamental Program logic, the actual algorithms for anomaly detection, meta-learning, or reinforcement learning, was either missing or incorrect.

This pattern confirms that **OpenAI o4-mini-high** is capable of scaffolding a Project's structure but fails to deliver on the complex, domain-specific logic, rendering its solutions incomplete for any practical use without major developer intervention.

FinTech (Python)



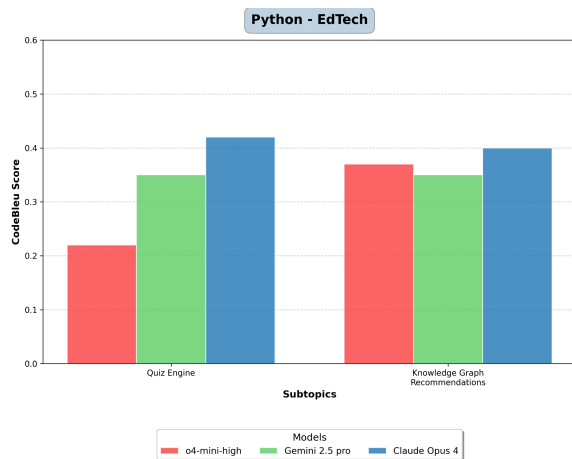
Claude Opus 4 consistently delivered the most robust, Production-ready, and complete solutions, which is reflected in its superior **CodeBLEU scores**. **Gemini 2.5 Pro** performed strongly, often Providing correct and well-structured code that met the Prompt's requirements, though sometimes with minor gaps. **OpenAI o4-mini-high** typically Produced functional Prototypes that served as a good starting point but regularly lacked the depth, security, and Production-level features required by the

Prompts, resulting in the lowest scores.

OpenAI o4-mini-high is consistently positioned as Providing a partial solution or a functional Prototype. While it correctly establishes a basic framework, it frequently fails to implement deeper technical requirements, Production-ready features, and critical security measures. Justifications mention that it lacks critical Production-ready features, uses simplistic implementations, and provides a sound structural baseline that requires significant additional work to be deployable.

- In the **FinTech microservice Prompt** (365), it had the lowest score of 0.30. This aligns perfectly with the justification that it omits several of the enterprise-critical requirements.
- For the **Monte Carlo Prompt** (366), its score of 0.26 reflects the justification that it lacks polish, testing, and Bloomberg integration, making it more of a Prototype.
- Its highest score came in the Fraud Detection Prompt (367) with 0.38, where the justification noted it delivered a comprehensive, Production-grade FastAPI application but with minor technical caveats, suggesting it performed better on this specific task but still fell short of **Claude's** more polished output.

EdTech (Python)

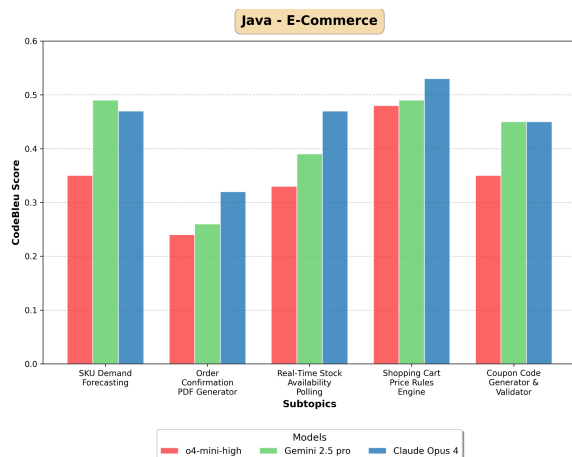


Claude Opus 4 and Gemini 2.5 Pro consistently delivered more mature, well-engineered, and feature-complete solutions. This is reflected in both the qualitative justifications and their generally higher **CodeBLEU scores**. **Claude's** solutions were often described as the most sophisticated, though with occasional inaccuracies. Gemini Provided solid, near-Production-ready code. **OpenAI o4-mini-high** had a more mixed performance; it produced a minimal prototype for

the more complex Prompt but delivered a surprisingly accurate algorithmic solution for the second Prompt, despite lacking polish.

- **OpenAI o4-mini-high** performance was inconsistent across the two Prompts. For the complex quiz engine, it Provided a **conceptual** and **minimal implementation** that was a good Prototype but limited in **immediate real-world readiness**. In sharp contrast, for the algorithm-focused learning pathway Prompt, it was praised for its **technically accurate** and **efficient implementation** that **precisely** aligns with the Prompt's **requirements**, although it lacked developer-friendly features like structured output. The **CodeBLEU scores** clearly reflect this performance variance. In the adaptive quiz Prompt, it had the lowest score of 0.22, which perfectly aligns with the justification that it Produced a **conceptual and minimal** Prototype lacking Production robustness.
- However, in the learning pathway, it achieved a score of 0.37, higher than Gemini's. The justification explains this: despite its lack of polish, it was the most accurate in implementing the required algorithms (Personalized PageRank and a true min-cost flow model), demonstrating strong core technical correctness.

E-Commerce (Java)

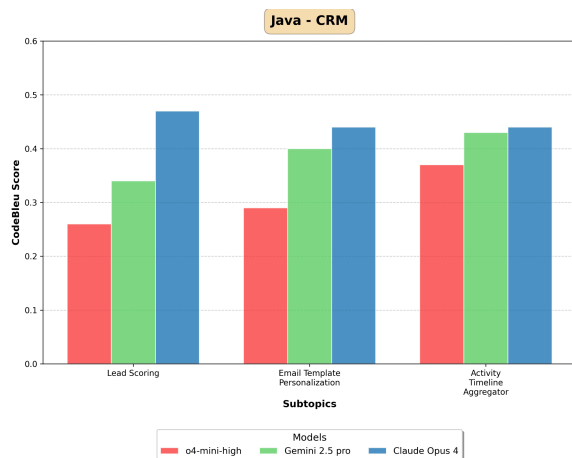


Claude Opus 4 consistently delivered the most comprehensive, feature-rich, and thoughtfully engineered solutions, which is reflected in its strong **CodeBLEU scores**. **Gemini 2.5 Pro** also performed at a very high level, often producing polished, modern, and Production-ready code that sometimes scored even higher than **Claude's**. **OpenAI o4-mini-high** reliably Produced functional and correct code that met the core requirements of the Prompts, but it generally lacked the advanced features, architectural polish, and developer-friendly

extras of the other two models.

- **OpenAI o4-mini-high** consistently delivers functional solutions that are "solid and reliable" and meet all the core Prompt requirements. However, its implementations often lack the sophistication, modern idioms, and robustness of the other models. Justifications describe its code as a **practical, drop-in solution** or a **fully functional** app that could benefit from being **more modular** or having richer documentation, making it a good baseline but not a standout example.
- In the Demand Prediction Prompt (370), its score of **0.35** is respectable and aligns with the description of a **well-written and functional solution** that could be more polished.
- For the PDF Generation Prompt (371), it had the lowest score of **0.24**. This matches the justification that it uses **older Java idioms** and its image handling is **rudimentary**, limiting its quality compared to others.
- Its score of **0.48** on the Rule Engine Prompt (373) was its strongest performance, nearly matching the others. This is supported by a positive justification stating it Provided a **correct, relevant, and complete solution** with only **minor issues**.

CRM (Java)

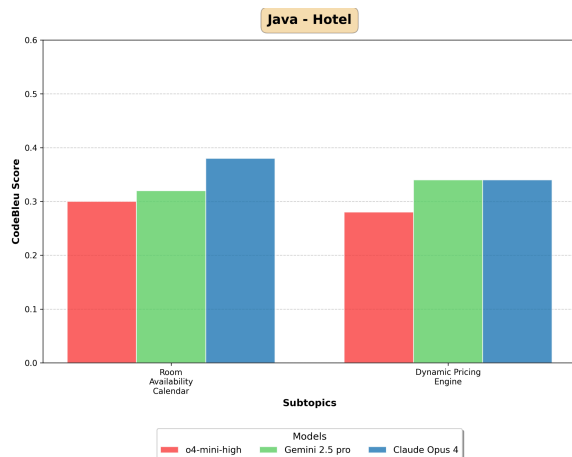


Claude Opus 4 and **Gemini 2.5 Pro** consistently delivered the most advanced, Professional, and feature-rich solutions, suitable for enterprise-level applications. Their strong performance is reflected in both the qualitative justifications and their leading **Codebleu scores**. **OpenAI o4-mini-high** reliably Produced functional and correct solutions that met the core requirements but generally lacked the architectural sophistication, scalability, and depth of the other two models, positioning it as a solid but

more basic option.

- **OpenAI o4-mini-high** consistently provides **solid, practical**, and functional solutions that serve as a **textbook example** of the Prompt's core requirements. Its code is clear and meets the immediate needs of the task. However, it regularly **falls short** on deeper requirements like compliance, integration depth, and advanced features, making its solutions good baselines but less powerful or extensible than the alternatives.
- In the Lead Scoring Prompt (375), it had the lowest score of **0.26**. This aligns perfectly with the justification that it "falls short in fully addressing compliance requirements and CRM integration depth.
- For the Email Template Prompt (376), its score of **0.29** was also the lowest, reflecting the description of a **solid, practical microservice** that follows standard patterns but lacks **novel ideas** or advanced features.
- In the Customer Timeline Prompt (377), its score of **0.37** was again the lowest, supporting the justification that it **covers the basics** and is **easy to use** but lacks the advanced features and scalability of the other two models.

Hotels (Java)

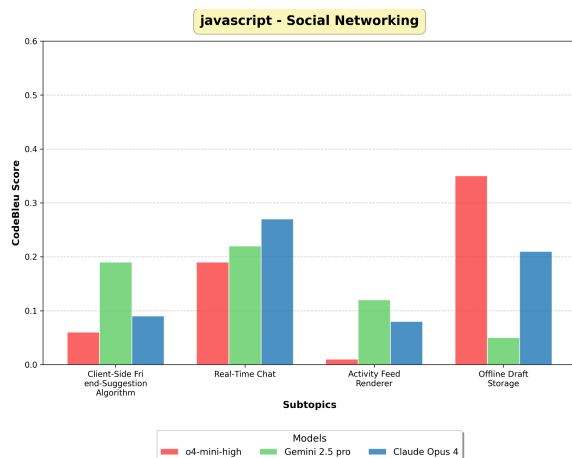


Claude Opus 4 and **Gemini 2.5 Pro** consistently stood out for delivering exceptionally robust, modern, and well-architected code, often earning perfect or near-perfect praise in the justifications. **OpenAI o4-mini-high** also performed excellently, providing complete and correct implementations that, while technically sound, were sometimes seen as less innovative or excellent maintainability. The justifications highlight its

ability to Produce code that is not only correct but also architecturally sound for large-scale platforms.

- **OpenAI o4-mini-high** is consistently positioned as Providing **excellent, technically sound**, and Production-quality implementations that correctly meet all Prompt requirements. Its code is **clear, practical, and highly usable**. However, the justifications also note that its solutions, while complete, sometimes lack the advanced **modularization, innovation**, or sophisticated architecture of the other two, making them a solid baseline rather than a standout example.
- The **CodeBLEU scores** reflect its status as a highly competent but slightly less advanced option compared to the others.
- In the Room Availability **Calendar Prompt** (378), it had the lowest score of **0.30**. This aligns with the justification that while it is a "complete, and well-structured" solution, it lacks the advanced features of the others.
- For the **Dynamic Pricing Engine Prompt** (379), its score of **0.28** was also the lowest. This supports the justification that while the implementation was "high-quality," its core pricing algorithm was "somewhat simplistic" compared to the more sophisticated systems from **Gemini 2.5 Pro** and **Claude Opus 4**.

Social Networking (JavaScript)

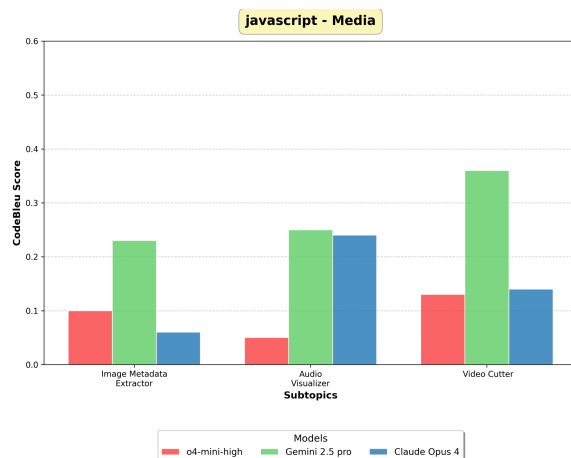


Claude Opus 4 and **Gemini 2.5 Pro** consistently delivered more polished, feature-rich, and Professionally engineered solutions. **Claude** often stood out for its completeness and user-centric design, while Gemini was praised for its clean architecture and modern practices. **OpenAI o4-mini-high** generally Produced functional but more basic implementations that met the core requirements but lacked the

advanced features and polish of the others. The **Codebleu scores** were sometimes inconsistent with the qualitative justifications, particularly for the Offline Draft Storage Prompt (393), highlighting the limitations of automated metrics in capturing user experience and architectural quality.

- **OpenAI o4-mini-high** consistently provides **solid, practical**, and functional solutions that meet the core requirements but are often **minimalist** or lack depth. Its implementations serve as a good foundation but often miss advanced features, sophisticated error handling, and the polished user experience provided by the other models. Justifications describe its work as a **functional but not fully Production-ready plugin** or a **minimalist, developer-focused approach**.
- Its scores on the first three Prompts were the lowest of the group: **0.06** (Mobile App), **0.19** (Chat), and **0.01** (Video Plugin). This aligns perfectly with the justifications describing its solutions as foundational but incomplete or less robust.
- However, on the Offline Draft Storage Prompt (393), it received the highest score of **0.35**. This is a **significant contradiction** to the justification that described its solution as a **minimalist technical Proof-of-concept that lacks essential modern features**. The high score strongly suggests its simple, direct implementation was a very close structural match to the reference solution, rewarding its simplicity over the more feature-rich but complex solutions from the other models.

Media (JavaScript)

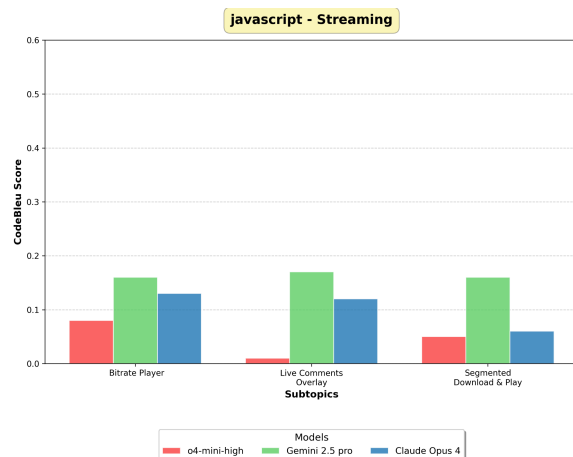


Gemini 2.5 Pro consistently produced the highest-scoring solutions, often praised for being technically sound and well-structured, although it sometimes failed to meet core requirements or had critical flaws despite its high scores. **Claude Opus 4** and **OpenAI o4-mini-high** had more mixed results; both were capable of producing functional or even sophisticated code, but also delivered solutions with significant technical

deviations, incompleteness, or basic implementations that lacked polish. The **Codebleu scores** for this set frequently contradicted the qualitative justifications, highlighting the limitations of automated metrics in capturing practical usability and adherence to specific technical constraints.

- **OpenAI o4-mini-high** consistently Produced functional but limited solutions. Its code was described as doing the **bare minimum** with **rudimentary** UI/UX (394) or as a **solid, complete implementation** that still had clear limitations like manual beat sync and monolithic rendering methods (395). It met the basic requirements but lacked the polish and advanced features of a Production-ready application.
- In the Image Metadata Prompt (394), it had a low score of **0.10**, which supports the justification that it **does the bare minimum** and **lacks creativity**.
- For the Audio Visualizer Prompt (395), its score of **0.05** was the lowest by a large margin, reflecting the justification's criticism of its simplistic beat detection and basic visuals.
- In the Video Cutter Prompt (396), its score of **0.13** was again low, which is consistent with the justification describing it as a **solid solution** but with the significant flaw of using the wrong underlying technology (mp4box.js instead of MSE)

Streaming (JavaScript)



Gemini 2.5 Pro consistently emerged as the top performer, delivering solutions that were frequently described as the "golden standard." Its code was praised for being complete, correct, and Production-ready. **Claude Opus 4** and **OpenAI o4-mini-high** both produced functional but less complete solutions. **Claude's** outputs were often more advanced and creative but sometimes had correctness issues, while **OpenAI o4-mini-high**

Provided solid but more basic foundations with notable gaps. In this domain, the **Codebleu scores** aligned very well with the qualitative justifications, clearly ranking Gemini first and **OpenAI o4-mini-high** last.

- **OpenAI o4-mini-high** consistently delivers solutions that are described as a "solid foundation" or a "practical starting point," but they consistently have significant functional gaps. Justifications note that its solutions lack key features like Proper HLS parsing, resumable playback, or robust error recovery, making them effective but incomplete and less suitable for Production compared to the others.
- The **Codebleu scores** consistently place **OpenAI o4-mini-high** as the weakest performer, which is in perfect agreement with the justifications.
- In the **ABR Player Prompt** (397), it had the lowest score of **0.08**. This aligns with the justification stating it is "effective but lacks seeking and real HLS parsing."
- For the **Comments Overlay Prompt** (398), its score of **0.01** was the lowest by a significant margin, reflecting the description of a solution with "gaps in HLS/DASH integration and edge case handling."
- In the **Advanced Streaming Module Prompt** (399), its score of **0.05** was again the lowest, supporting the justification that it had "minor gaps in resumable playback, checksum validation, and error handling."

Conclusion

Language	Domain/Industry	Winning Model(s)	Key Strengths/Notes
C++	Gaming	Claude Opus 4	High completeness and correctness, excellent physics and game mechanics implementations
C++	IoT	Claude Opus 4 & OpenAI o4-mini-high (tie)	OpenAI o4-mini-high edged out on Relevance and Correctness, Claude Opus 4 led in Completeness
C++	Security	Claude Opus 4	Best in producing sophisticated, secure, production-ready implementations
Python	Machine Learning	Claude Opus 4	Superior technical correctness, strong relevance, and advanced implementation
Python	FinTech	Claude Opus 4 & OpenAI o4-mini-high (tie)	Claude offers advanced complete solutions; OpenAI consistently aligns with core requirements
Python	EdTech	Claude Opus 4	Exceptional completeness and correctness, consistently covers all requirements comprehensively
Java	E-Commerce	Gemini 2.5 Pro	Robustness and correctness with advanced architecture and enterprise features
Java	CRM	Gemini 2.5 Pro	Flawless alignment, robust architecture, comprehensive CRM implementations
Java	Hotel	Gemini 2.5 Pro & OpenAI o4-mini-high (tie)	Complete solutions, reliable implementations, consistently correct business logic
JavaScript	Social Networking	Gemini 2.5 Pro	Highest relevance, completeness, and correctness, best user-centric implementations
JavaScript	Media	Gemini 2.5 Pro	Technically sound, highest overall correctness, superior architecture
JavaScript	Streaming	Gemini 2.5 Pro	Flawless correctness, exceptional comprehensive and production-ready solutions

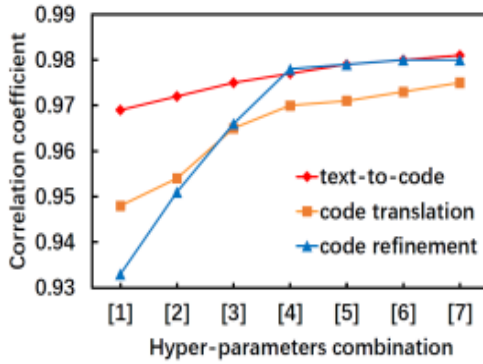
References

- [1] LiveCodeBench: <https://livecodebench.github.io/leaderboard.html>.
- [2] LLMarena: <https://lmarena.ai/leaderboard>
- [3] Ren, Shuo, et al. "CodeBLEU: a method for automatic evaluation of code synthesis." arXiv preprint arXiv:2009.10297 (2020). <https://arxiv.org/pdf/2009.10297>
- [4] Papineni, K.; Roukos, S.; Ward, T.; and Zhu, W.-J. 2002. BLEU: a method for automatic evaluation of machine translation. In Proceedings of the 40th annual meeting of the Association for Computational Linguistics. <https://aclanthology.org/P02-1040.pdf>

Appendix

CodeBLEU is a composite metric designed specifically for code generation tasks. It evaluates the similarity between the generated code and the reference (golden) code using multiple dimensions:

$$\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{BLEU}_{\text{weight}} + \gamma \cdot \text{MatchAST} + \delta \cdot \text{MatchDF}$$



Combination	$\alpha, \beta, \gamma, \delta$
[1]	0.40, 0.40, 0.10, 0.10
[2]	0.35, 0.35, 0.15, 0.15
[3]	0.30, 0.30, 0.20, 0.20
[4]	0.25, 0.25, 0.25, 0.25
[5]	0.20, 0.20, 0.30, 0.30
[6]	0.15, 0.15, 0.35, 0.35
[7]	0.10, 0.10, 0.40, 0.40

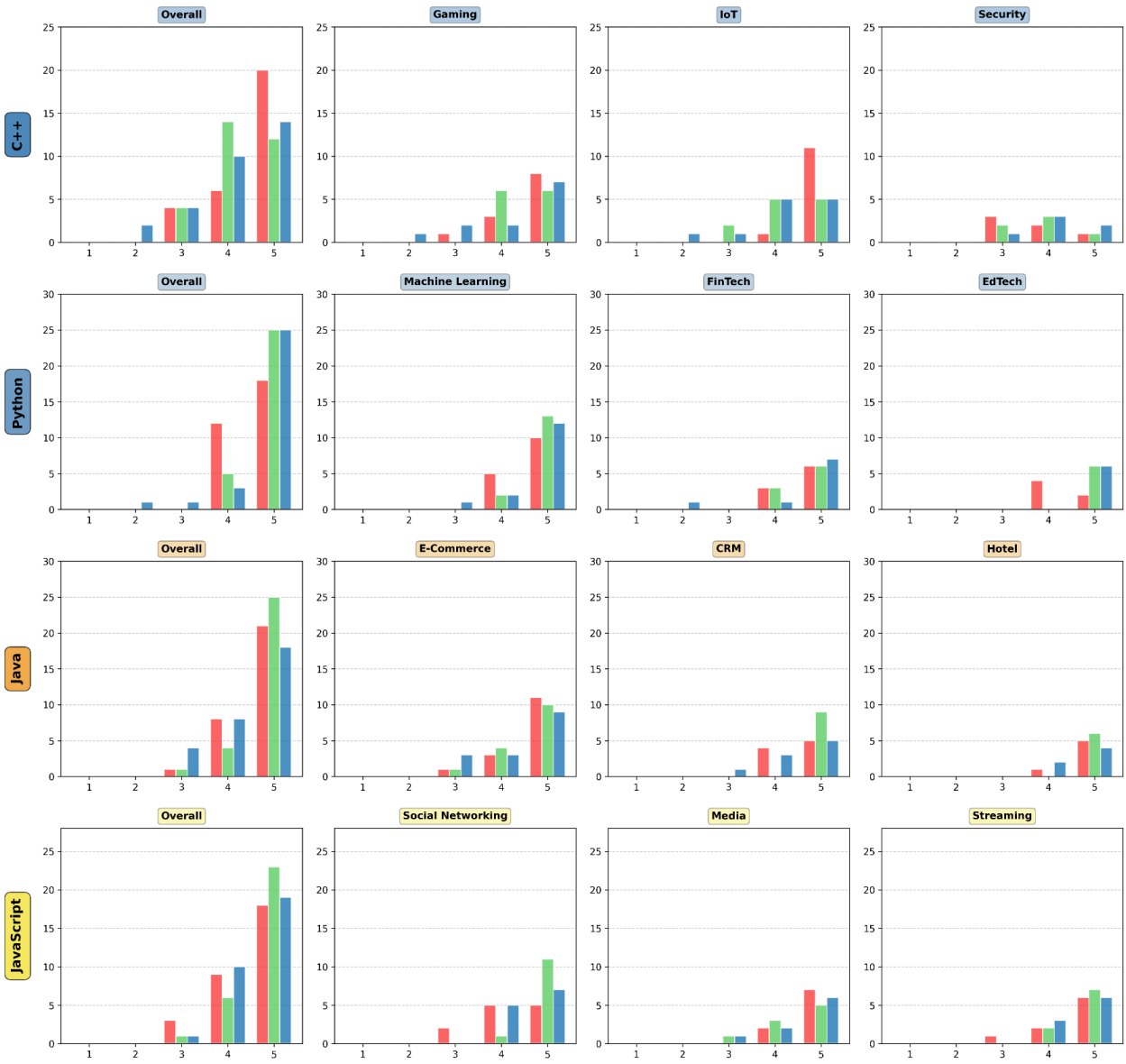
Figure 2a and 2b are taken from paper [3]

The performance tends to stabilize after combination [4], with combination [7], where $\alpha, \beta, \gamma, \delta = 0.1, 0.1, 0.4, 0.4$, achieving the highest overall results among all configurations in Figure 2a (0.981, 0.975, and 0.980 for the three tasks, respectively). However, [7] is not universally optimal. For instance, the configuration $\alpha, \beta, \gamma, \delta = 0.1, 0.4, 0.1, 0.4$ yields a better correlation coefficient (0.984) than [7] (0.980) for the code refinement task. Nevertheless, we recommend using combination [7] for general code synthesis tasks, as the last two components (γ and δ) appear to have a stronger correlation with human evaluation scores, as suggested by Table 4.

The score interpretation is:

1. A **score close to 1.0** indicates high similarity in structure, logic, and syntax.
2. A **score above 0.5** is generally considered acceptable in many code generation tasks.
3. A **low score (e.g., < 0.3)** may indicate syntactic or semantic divergence from the reference.

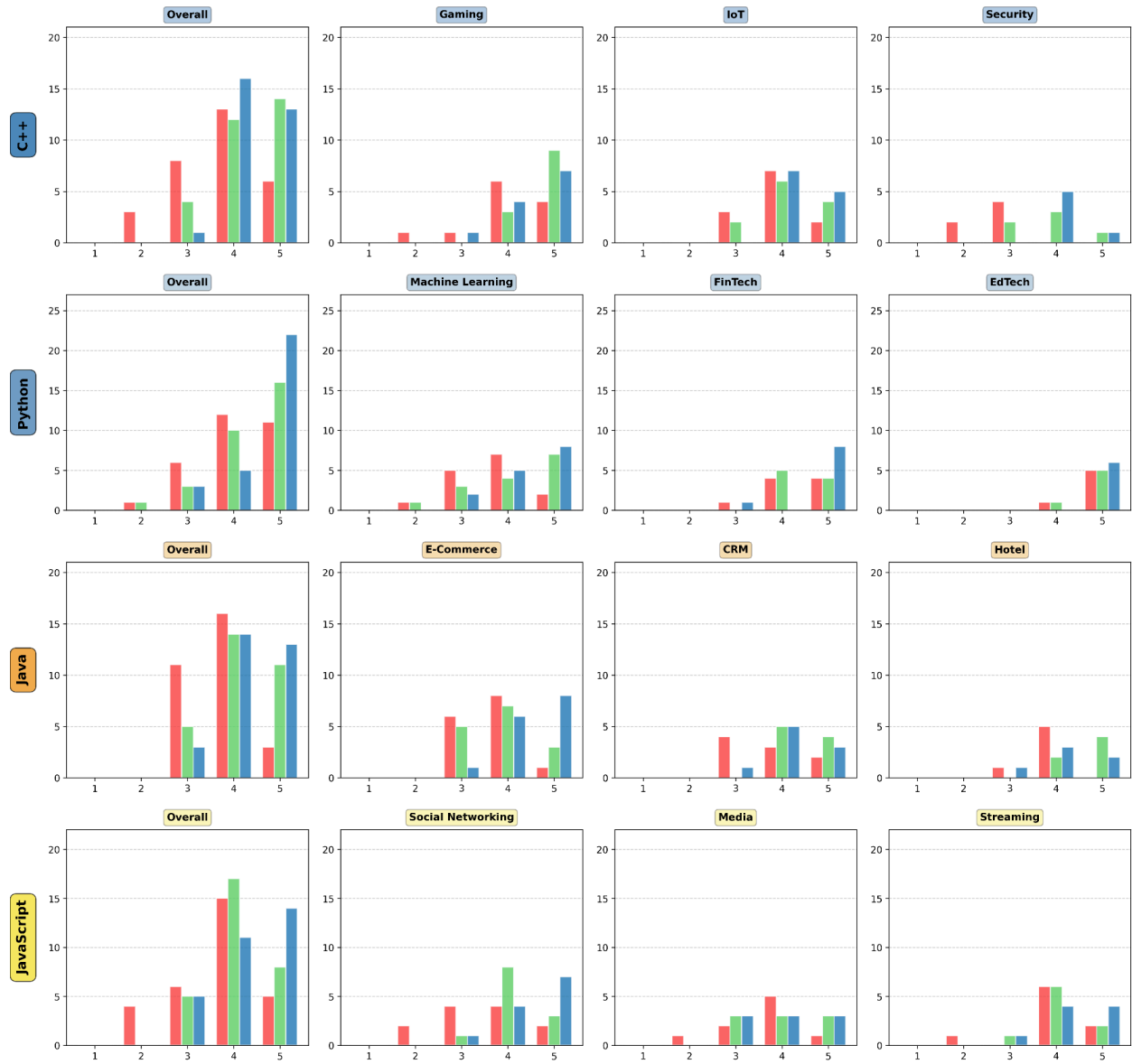
Coherence Ratings - Cross-Language Domain Comparison



Models
o4-mini-high Gemini 2.5 pro Claude Opus 4

Chart Guide
X-Axis: Rating (1-5) Y-Axis: Number of Tasks

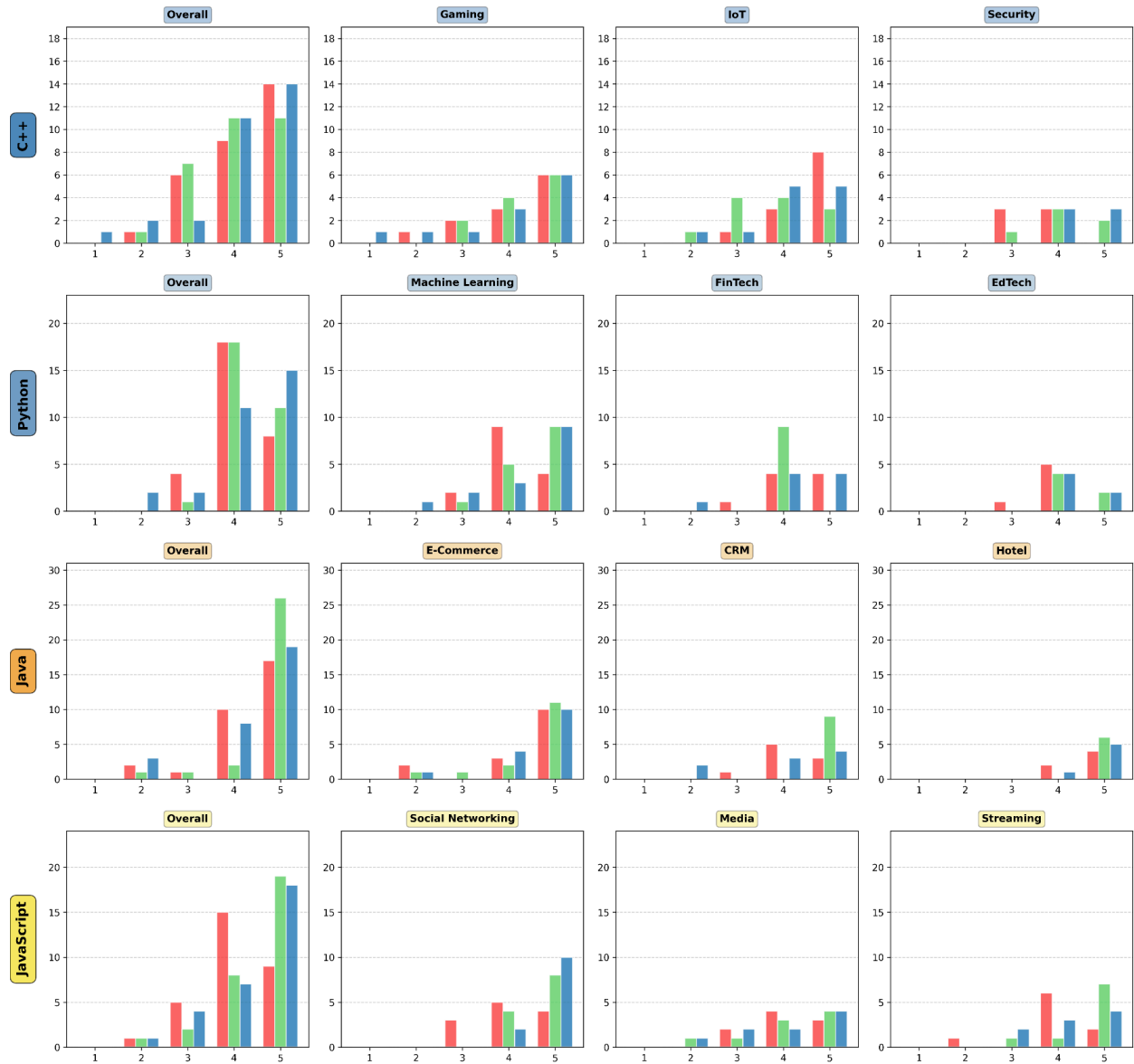
Creativity Ratings - Cross-Language Domain Comparison



Models
o4-mini-high Gemini 2.5 pro Claude Opus 4

Chart Guide
X-Axis: Rating (1-5) Y-Axis: Number of Tasks

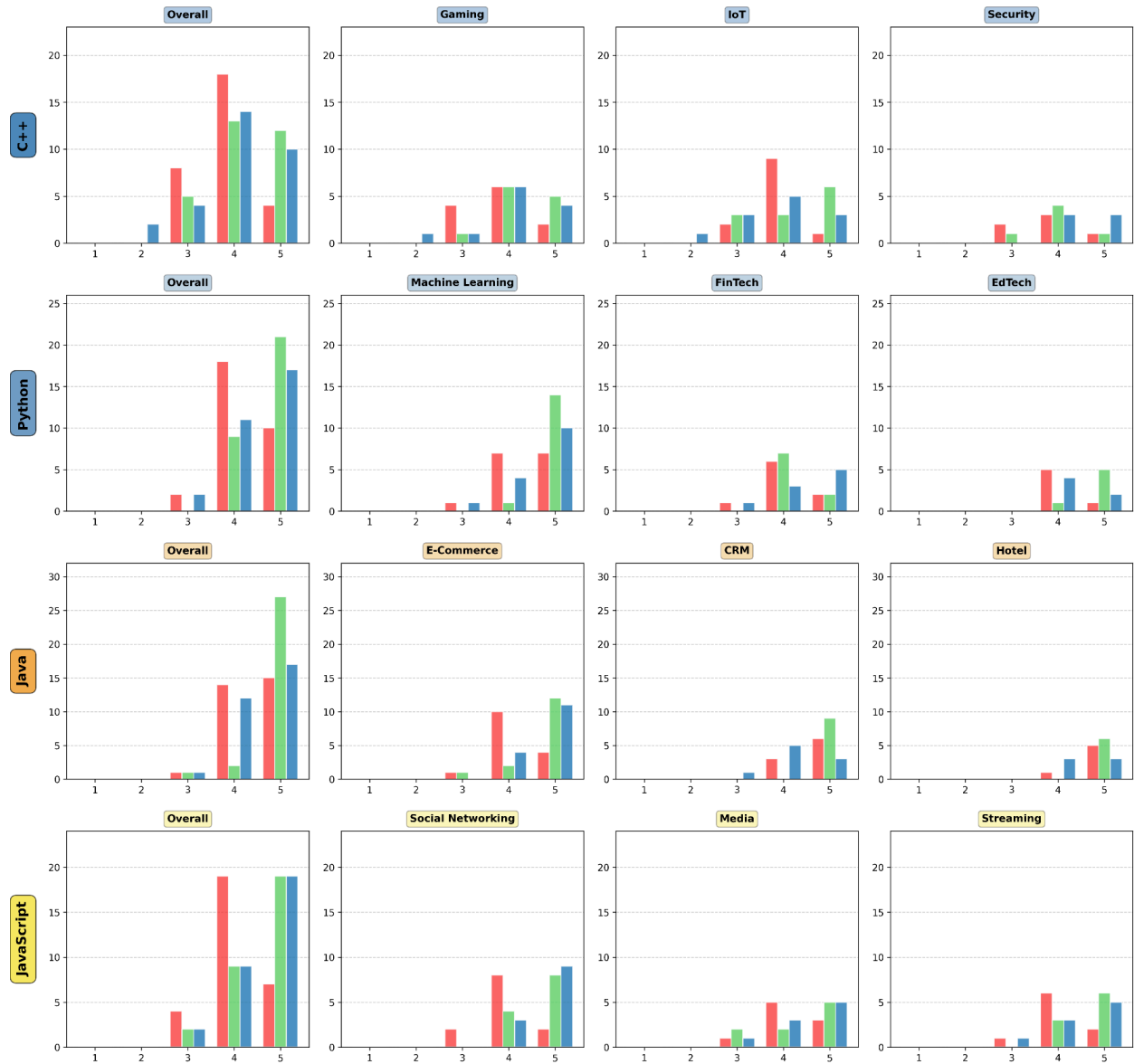
Helpfulness Ratings - Cross-Language Domain Comparison



Models
o4-mini-high Gemini 2.5 pro Claude Opus 4

Chart Guide
X-Axis: Rating (1-5) Y-Axis: Number of Tasks

Style Presentation Ratings - Cross-Language Domain Comparison



Models
o4-mini-high Gemini 2.5 pro Claude Opus 4

Chart Guide
X-Axis: Rating (1-5) Y-Axis: Number of Tasks